Cache Efficient Data Structure

Memory: Fast, Large & Cheap!

For efficient computation we need

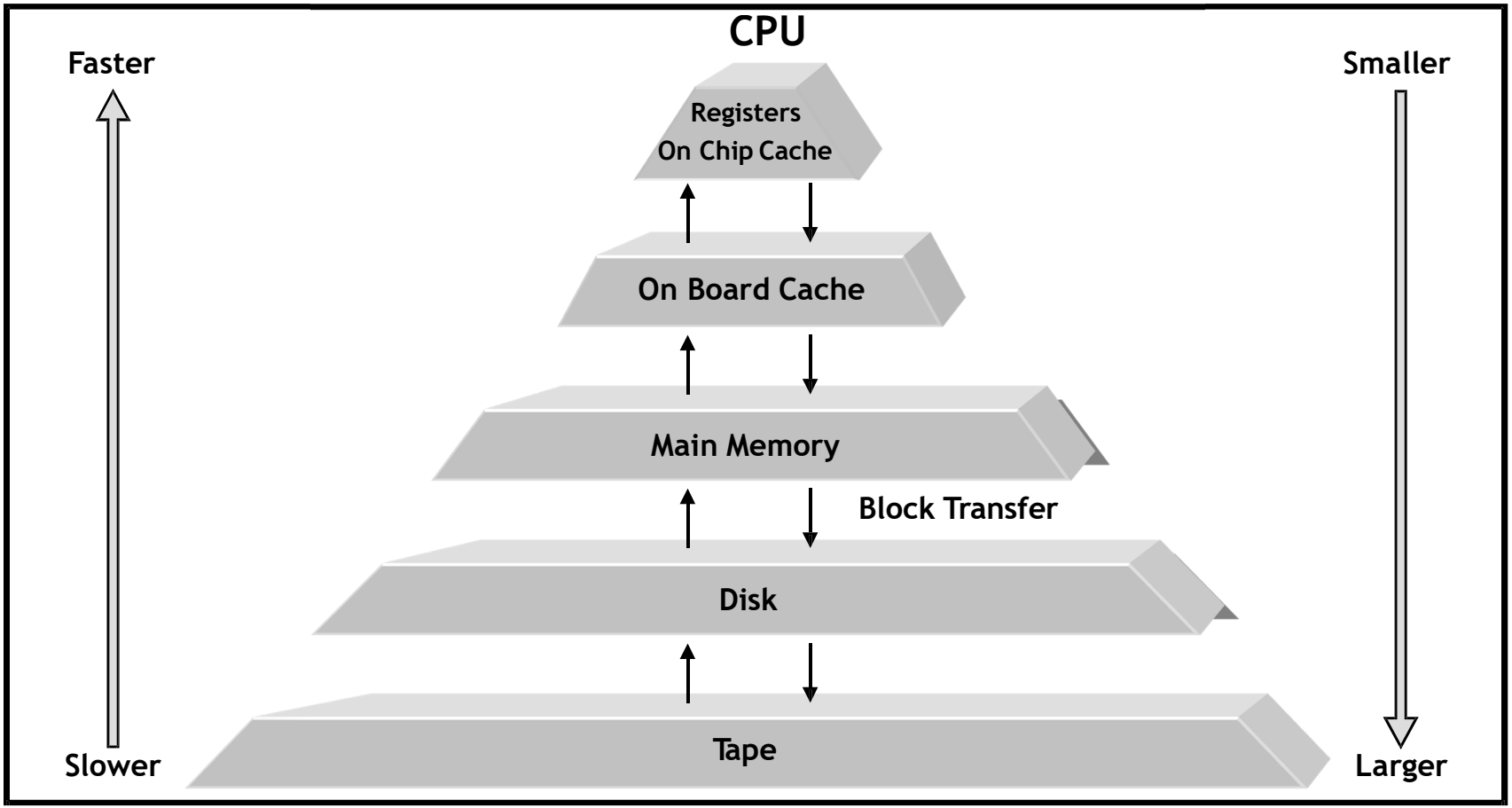
- fast processors
- fast and large (but not so expensive) memory

But memory cannot be cheap, large and fast at the same time, because of

- finite signal speed
- lack of space to put enough connecting wires

A reasonable compromise is to use a *memory hierarchy*.

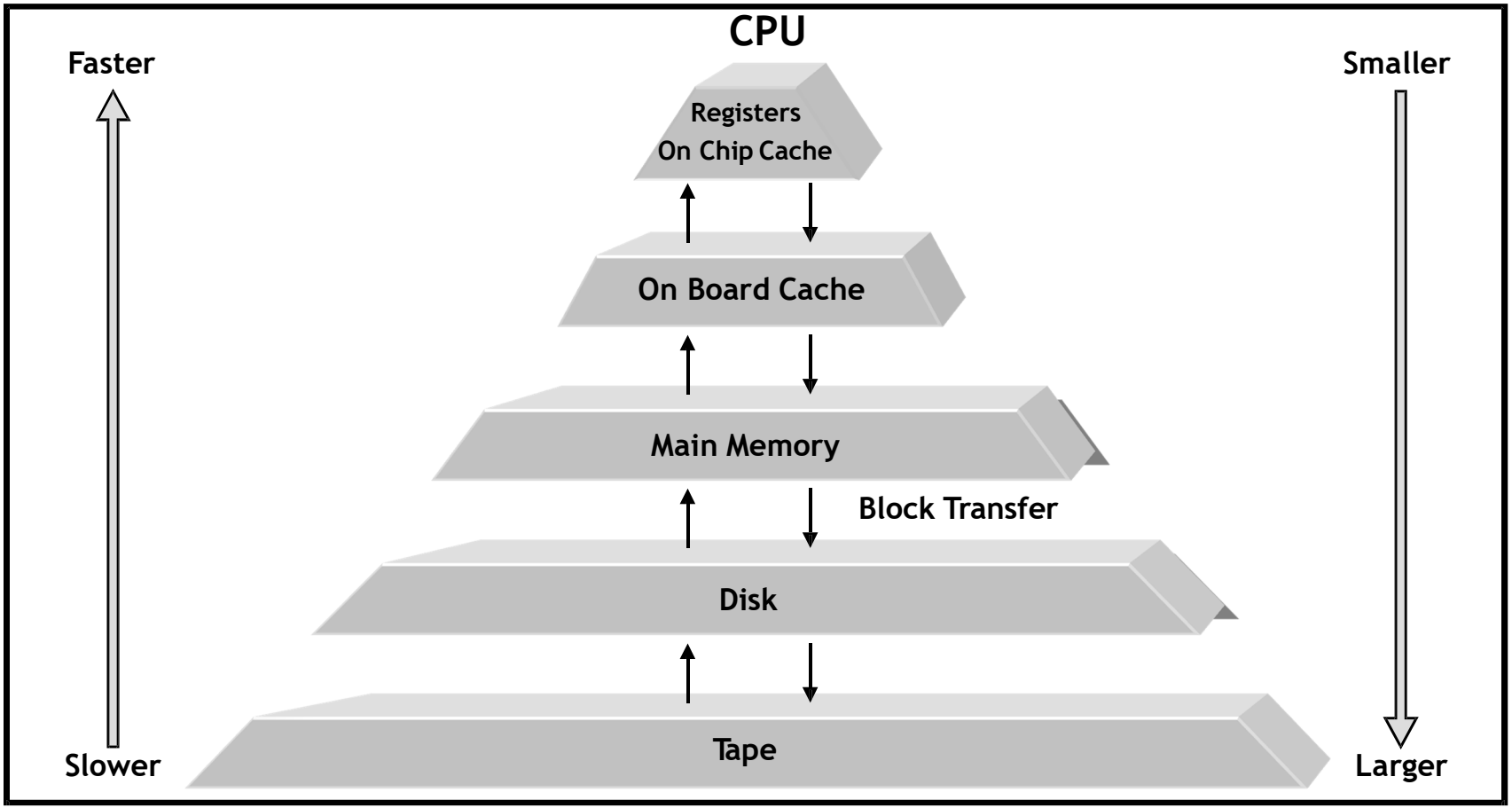
The Memory Hierarchy



A *memory hierarchy* is

- almost as fast as its fastest level
- almost as large as its largest level
- inexpensive

The Memory Hierarchy



To perform well on a memory hierarchy algorithms must have high locality in their memory access patterns.

Locality of Reference

Spatial Locality: When a block of data is brought into the cache it should contain as much useful data as possible.

Temporal Locality: Once a data point is in the cache as much useful work as possible should be done on it before evicting it from the cache.

CPU-bound vs. Memory-bound Algorithms

The Op-Space Ratio: Ratio of the number of operations performed by an algorithm to the amount of space (input + output) it uses.

Intuitively, this gives an upper bound on the average number of operations performed for every memory location accessed.

CPU-bound Algorithm:

- high op-space ratio
- more time spent in computing than transferring data
- a faster CPU results in a faster running time

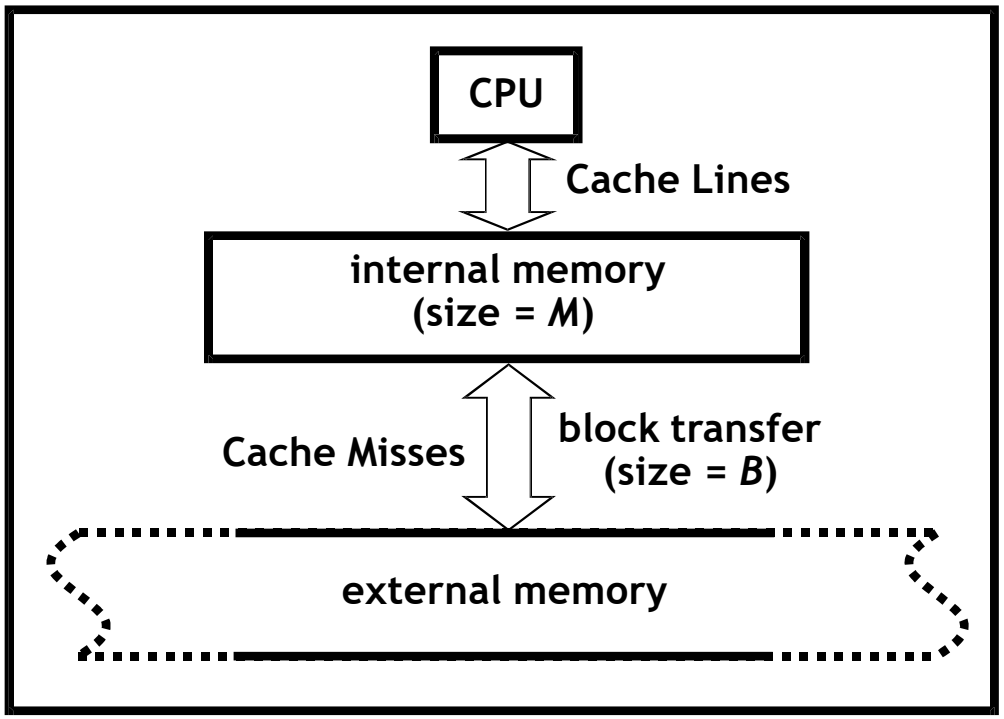
Memory-bound Algorithm:

- low op-space ratio
- more time spent in transferring data than computing
- a faster memory system leads to a faster running time

The Two-level I/O Model

The *two-level I/O model* consists of:

- an *internal memory* of size *M*
- an arbitrarily large *external memory* partitioned into blocks of size *B*.



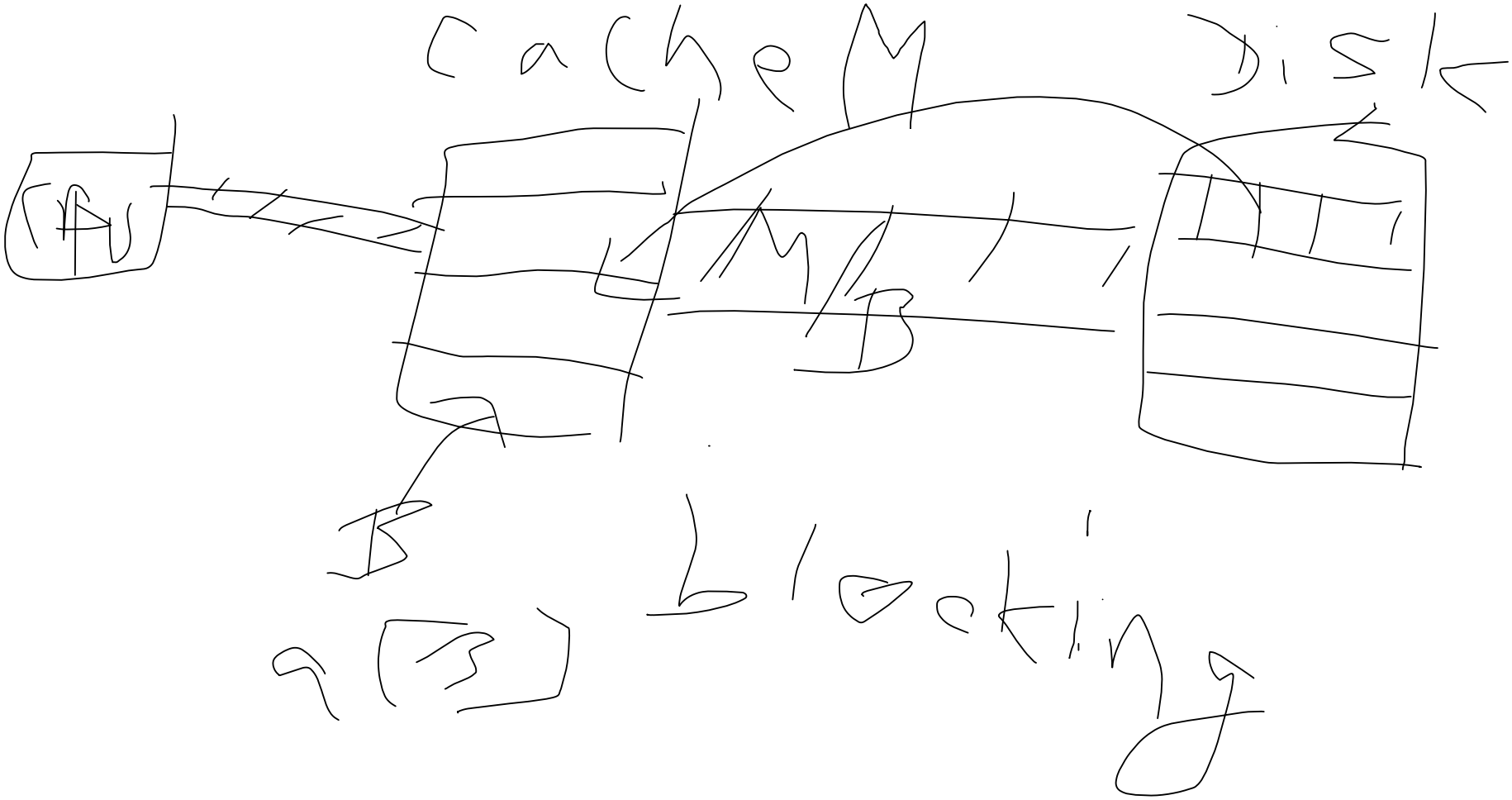
I/O complexity of an algorithm

= number of blocks transferred between these two levels

Basic I/O complexities: $\mathrm{scan}(N) = \Theta\left(\frac{N}{B}\right)$ and $\mathrm{sort}(N) = \Theta\left(\frac{N}{B}\log_{\frac{M}{B}}\frac{N}{B}\right)$

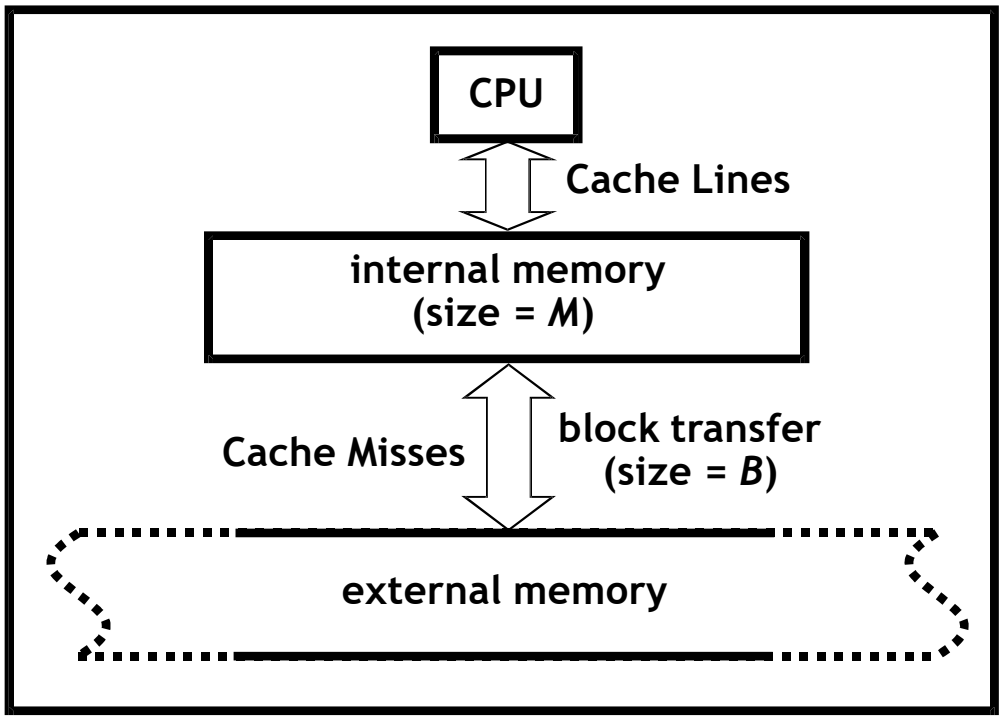
Algorithms often crucially depend on the knowledge of *M* and *B*

$\Rightarrow$ algorithms do not adapt well when *M* or *B* changes


cache
disk
CPU
MB
B
blocking

The Ideal-Cache Model

The *ideal-cache model* is an extension of the I/O model with the following constraint:



> algorithms are not allowed to use knowledge of *M* and *B*.

Consequences of this extension

- algorithms can simultaneously adapt to all levels of a multi-level memory hierarchy
- algorithms become more flexible and portable

Algorithms for this model are known as *cache-oblivious algorithms*.

The Ideal-Cache Model: Assumptions

The model makes the following assumptions:

- Optimal offline cache replacement policy
- Exactly two levels of memory
- Automatic replacement & full associativity

The Ideal-Cache Model: Assumptions

The model makes the following assumptions:

- Optimal offline cache replacement policy
  - LRU & FIFO allow for a constant factor approximation of optimal
- Exactly two levels of memory
- Automatic replacement & full associativity

The Ideal-Cache Model: Assumptions

The model makes the following assumptions:

- Optimal offline cache replacement policy
- Exactly two levels of memory
  - can be effectively removed by making several reasonable assumptions about the memory hierarchy
- Automatic replacement & full associativity

The Ideal-Cache Model: Assumptions

The model makes the following assumptions:

- Optimal offline cache replacement policy
- Exactly two levels of memory
- Automatic replacement & full associativity
  - in practice, cache replacement is automatic
    (by OS or hardware)
  - fully associative LRU caches can be simulated in software
    with only a constant factor loss in expected performance

The Ideal-Cache Model: Assumptions

The model makes the following assumptions:

- Optimal offline cache replacement policy
- Exactly two levels of memory
- Automatic replacement & full associativity

Often makes the following assumption, too:

- $M = \Omega\left(B^2\right)$, i.e., the cache is *tall*

The Ideal-Cache Model: Assumptions

The model makes the following assumptions:

- Optimal offline cache replacement policy
- Exactly two levels of memory
- Automatic replacement & full associativity

Often makes the following assumption, too:

- $M = \Omega(B^2)$, i.e., the cache is *tall*
  - most practical caches are tall

The Ideal-Cache Model: I/O Bounds

Cache-oblivious vs. cache-aware bounds:

- Basic I/O bounds (same as the cache-aware bounds):
  - $\mathrm{scan}(N) = \Theta\left(\frac{N}{B}\right)$
  - $\mathrm{sort}(N) = \Theta\left(\frac{N}{B}\log_{\frac{M}{B}}\frac{N}{B}\right)$
- Most cache-oblivious results match the I/O bounds of their cache-aware counterparts
- There are few exceptions; e.g., no cache-oblivious solution to the *permutation* problem can match cache-aware I/O bounds

Some Known Cache Aware / Oblivious Results

Problem	Cache-Aware Results	Cache-Oblivious Results
Array Scanning (*scan(N)*)	$O\left(\frac{N}{B}\right)$	$O\left(\frac{N}{B}\right)$
Sorting (*sort(N)*)	$O\left(\frac{N}{B}\log_{\frac{M}{B}}\frac{N}{B}\right)$	$O\left(\frac{N}{B}\log_{\frac{M}{B}}\frac{N}{B}\right)$
Selection	$O(scan(N))$	$O(scan(N))$
B-Trees [Am] (*Insert*, *Delete*)	$O\left(\log_B \frac{N}{B}\right)$	$O\left(\log_B \frac{N}{B}\right)$
Priority Queue [Am] (*Insert*, *Weak Delete*, *Delete-Min*)	$O\left(\frac{1}{B}\log_{\frac{M}{B}}\frac{N}{B}\right)$	$O\left(\frac{1}{B}\log_{\frac{M}{B}}\frac{N}{B}\right)$
Matrix Multiplication	$O\left(\frac{N^3}{B\sqrt{M}}\right)$	$O\left(\frac{N^3}{B\sqrt{M}}\right)$
Single Source Shortest Paths	$O\left(\left(V+\frac{E}{B}\right)\cdot\log_2\frac{V}{B}\right)$	$O\left(\left(V+\frac{E}{B}\right)\cdot\log_2\frac{V}{B}\right)$
Minimum Spanning Forest	$O\left(\min\left(sort(E)\log\log_{\frac{E}{V}} V, E+sort(E)\right)\right)$	$O\left(\min\left(sort(E)\log\log_{\frac{E}{V}}\frac{VB}{E}, V+sort(E)\right)\right)$

Table 1: *N* = #elements, *V* = #vertices, *E* = #edges, Am = Amortized.

Matrix Multiplication

Iterative Matrix Multiplication

$$z_{ij} = \sum_{k=1}^{n} x_{ik} y_{kj}$$

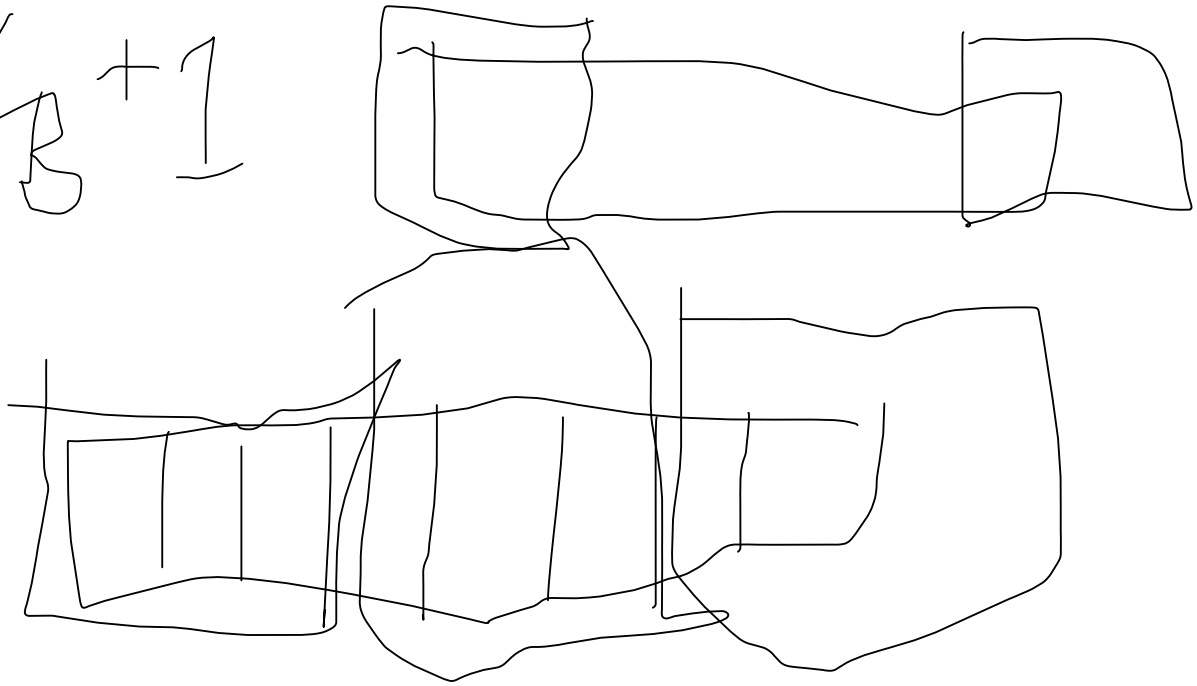
$$\begin{pmatrix} z_{11} & z_{12} & \cdots & z_{1n} \\ z_{21} & z_{22} & \cdots & z_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ z_{n1} & z_{n2} & \cdots & z_{nn} \end{pmatrix} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nn} \end{pmatrix} \times \begin{pmatrix} y_{11} & y_{12} & \cdots & y_{1n} \\ y_{21} & y_{22} & \cdots & y_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ y_{n1} & y_{n2} & \cdots & y_{nn} \end{pmatrix}$$

```
Iter-MM ( X, Y, Z, n)
1.  for  i ← 1  to  n  do
2.       for  j ← 1  to  n  do
3.            for  k ← 1  to  n  do
4.                 z_ij ← z_ij + x_ik × y_kj
```

Scan

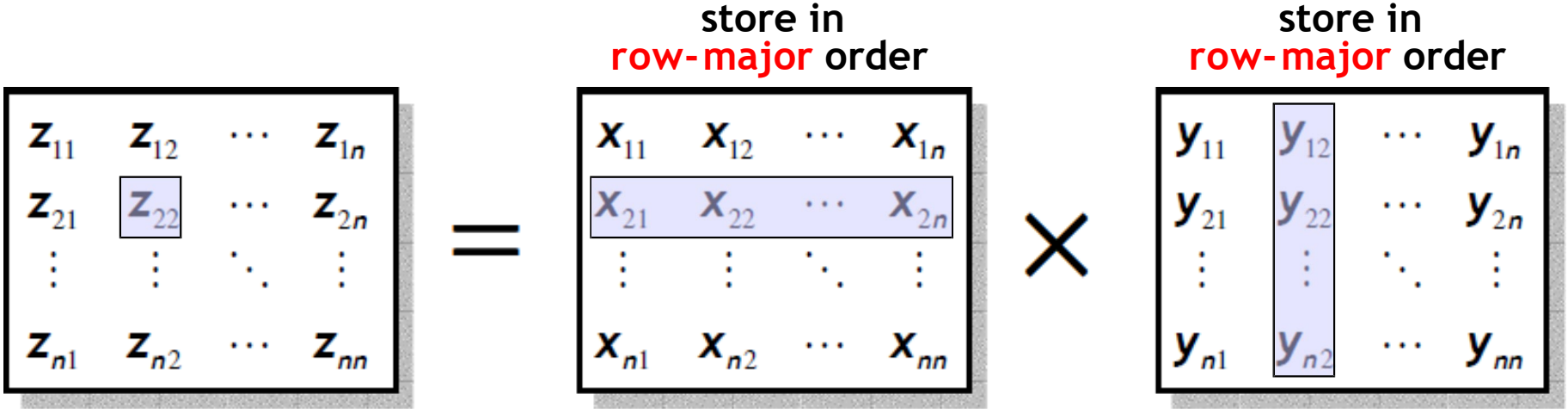
for - N/B + 1

```
for i in range(N/2):
    swap(A[i],A[N-i-1])
```



Iterative Matrix Multiplication

```
Iter-MM ( X, Y, Z, n )
1.  for i ← 1 to n do
2.      for j ← 1 to n do
3.          for k ← 1 to n do
4.              z_ij ← z_ij + x_ik × y_kj
```

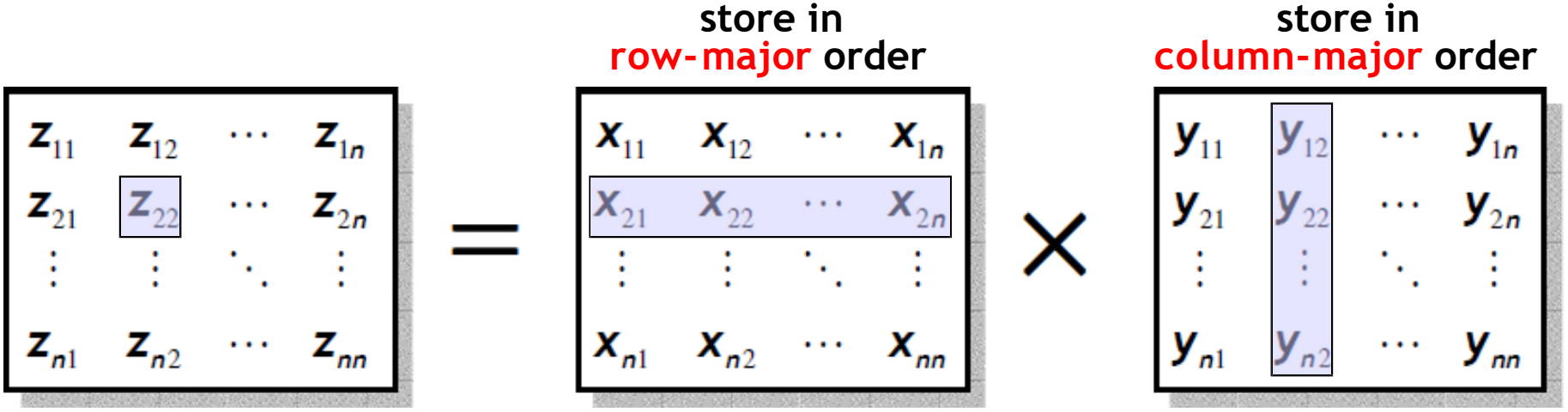


Each iteration of the *for* loop in line 3 incurs $O(n)$ cache misses.

I/O-complexity of *Iter-MM*, $Q(n) = O(n^3)$

Iterative Matrix Multiplication

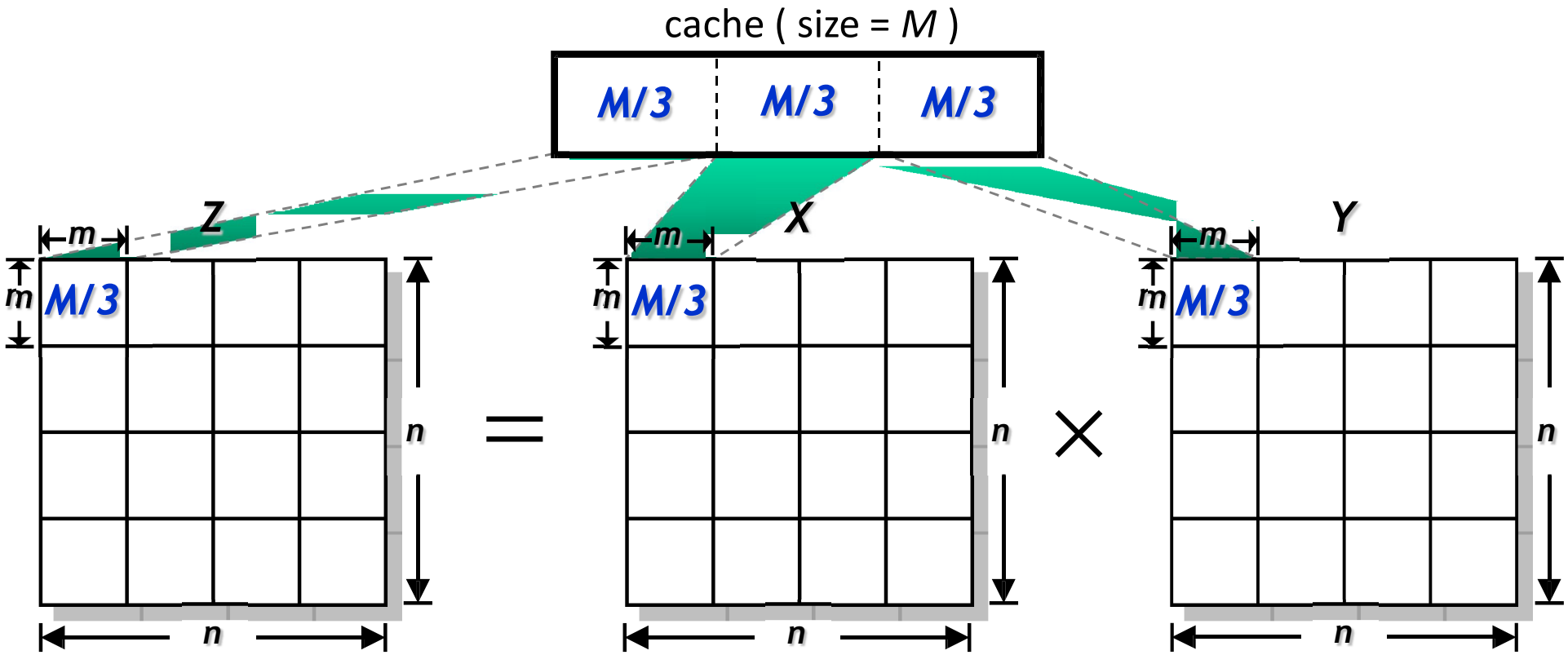
```
Iter-MM ( X, Y, Z, n )
1.  for i ← 1 to n do
2.      for j ← 1 to n do
3.          for k ← 1 to n do
4.              z_ij ← z_ij + x_ik × y_kj
```



Each iteration of the *for* loop in line 3 incurs $O\left(1+\frac{n}{B}\right)$ cache misses.

I/O-complexity of *Iter-MM*, $Q(n) = O\left(n^2\left(1+\frac{n}{B}\right)\right) = O\left(\frac{n^3}{B}+n^2\right)$

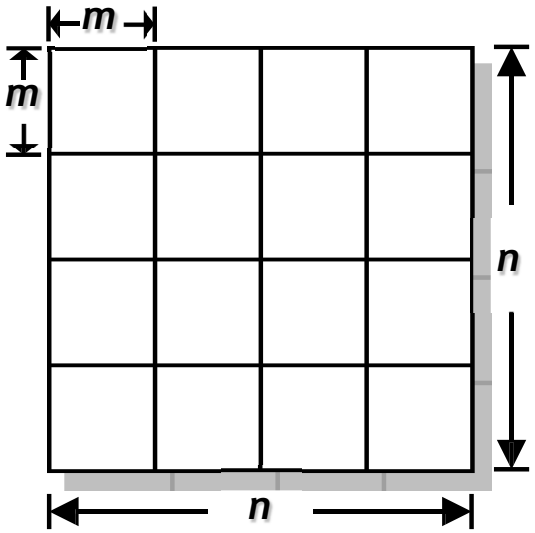
Block Matrix Multiplication




```
Block-MM( X, Y, Z, n )

1.  for i ← 1 to n / m  do
2.      for j ← 1 to n / m do
3.          for k ← 1 to n / m  do
4.              Iter-MM ( X_ik, Y_kj, Z_ij )
```

Block Matrix Multiplication




```
Block-MM ( X, Y, Z, n )
1.  for i ← 1 to n / m do
2.      for j ← 1 to n / m do
3.          for k ← 1 to n / m do
4.              Iter-MM ( X_ik, Y_kj, Z_ij )
```

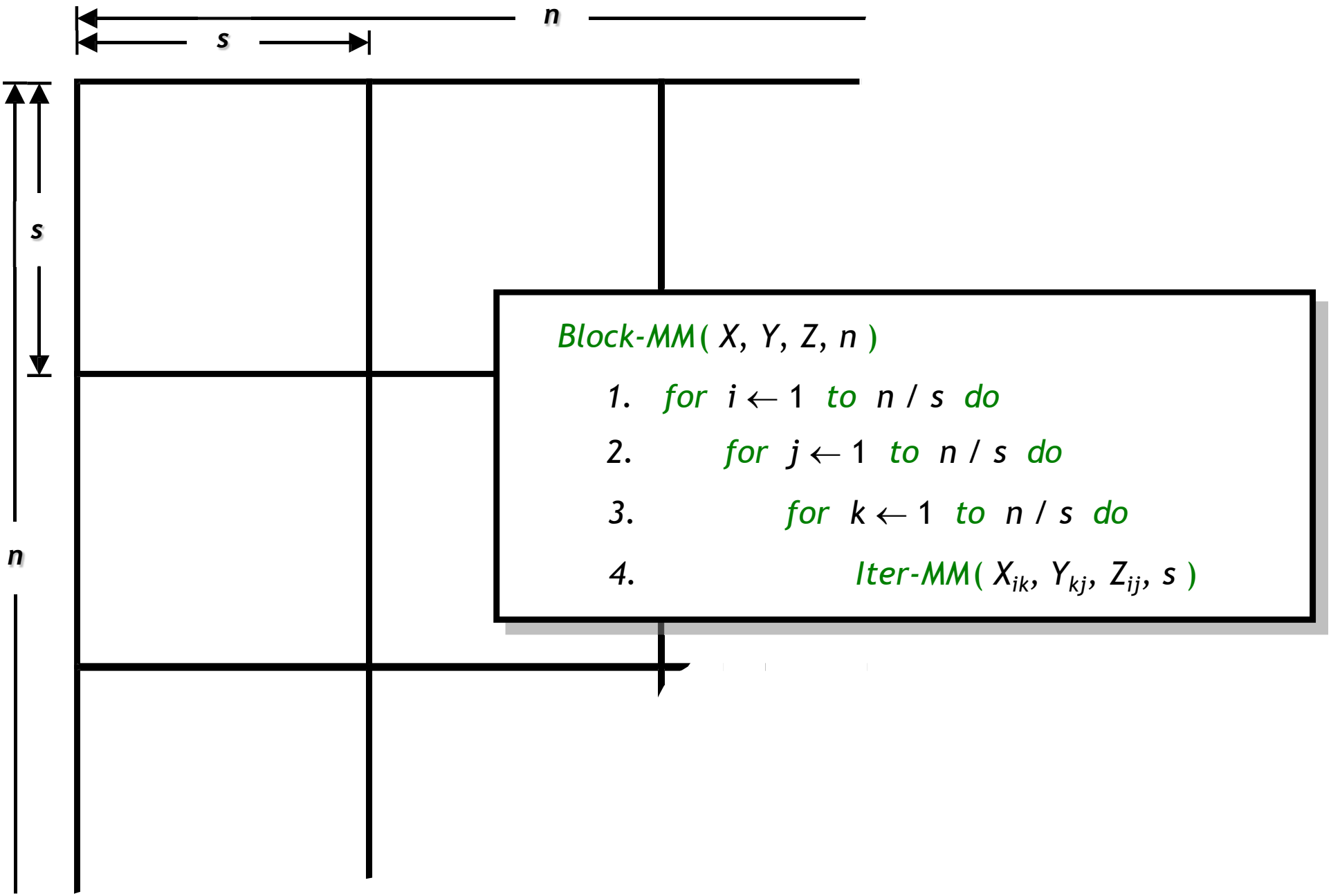
Choose $m = \sqrt{M/3}$, so that X_{ik}, Y_{kj} and Z_{ij} just fit into the cache.

Then line 4 incurs $\Theta\left(m\left(1+\frac{m}{B}\right)\right)$ cache misses.

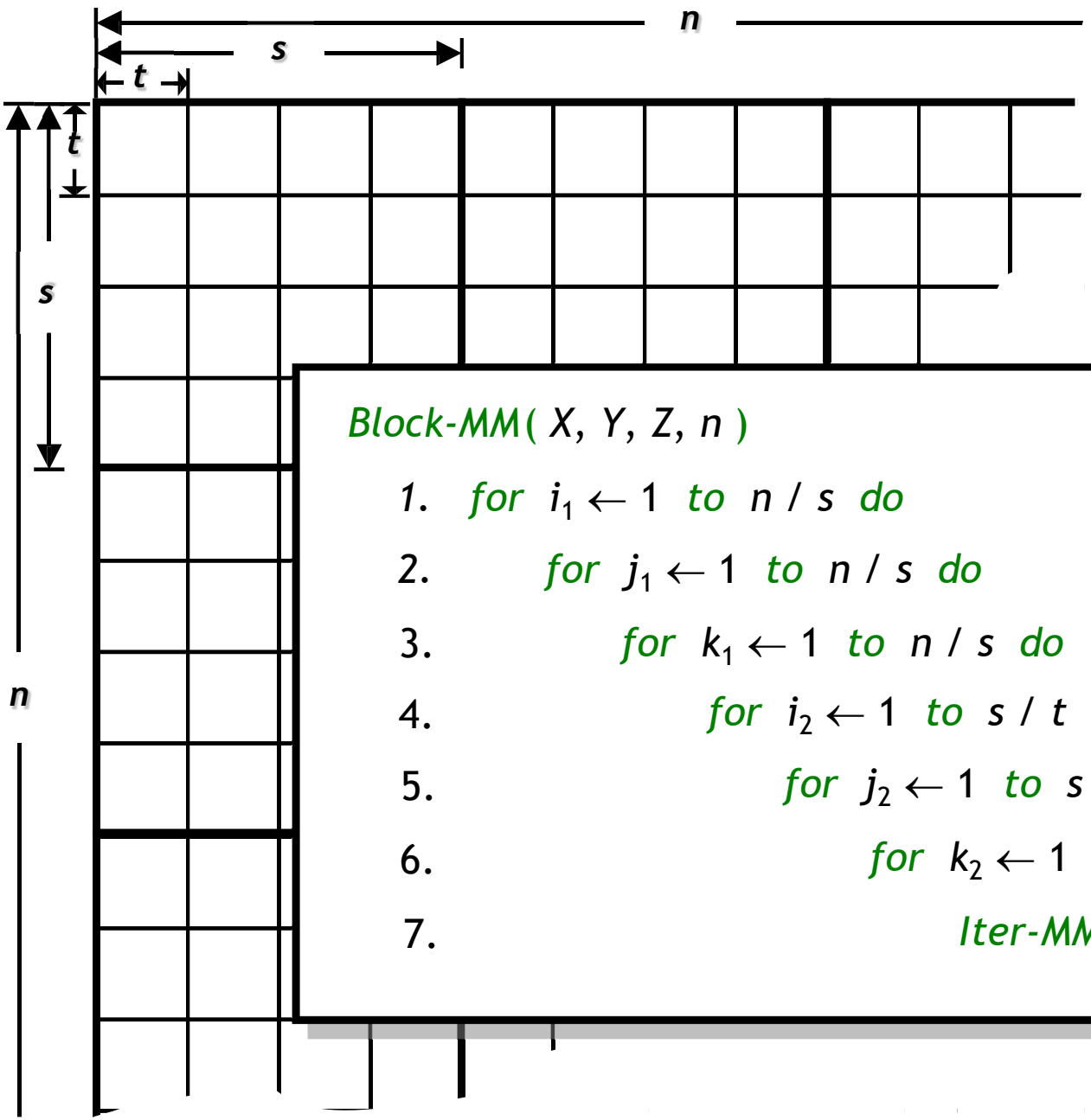
I/O-complexity of *Block-MM* [assuming a *tall cache*, i.e., $M = \Omega(B^2)$]

$$= \Theta\left(\left(\frac{n}{m}\right)^3\left(m + \frac{m^2}{B}\right)\right) = \Theta\left(\frac{n^3}{m^2} + \frac{n^3}{Bm}\right) = \Theta\left(\frac{n^3}{M} + \frac{n^3}{B\sqrt{M}}\right) = \Theta\left(\frac{n^3}{B\sqrt{M}}\right)$$

Multiple Levels of Cache

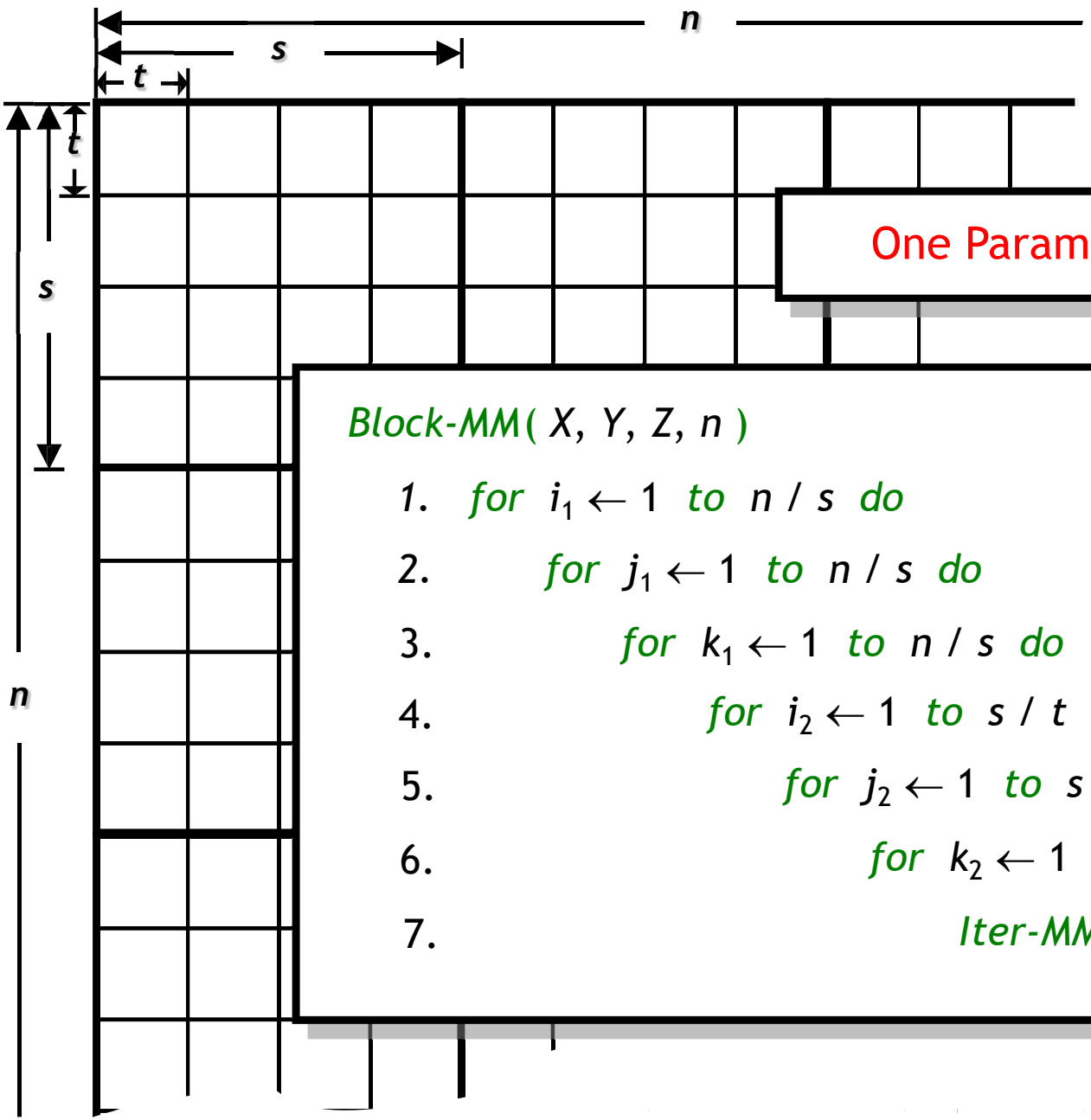


Multiple Levels of Cache




```
Block-MM( X, Y, Z, n )
1.  for i1 ← 1 to n / s do
2.      for j1 ← 1 to n / s do
3.          for k1 ← 1 to n / s do
4.              for i2 ← 1 to s / t do
5.                  for j2 ← 1 to s / t do
6.                      for k2 ← 1 to s / t do
7.                          Iter-MM( (X_{i1k1})_{i2k2}, (Y_{k1j1})_{k2j2}, (X_{i1j1})_{i2j2}, t )
```

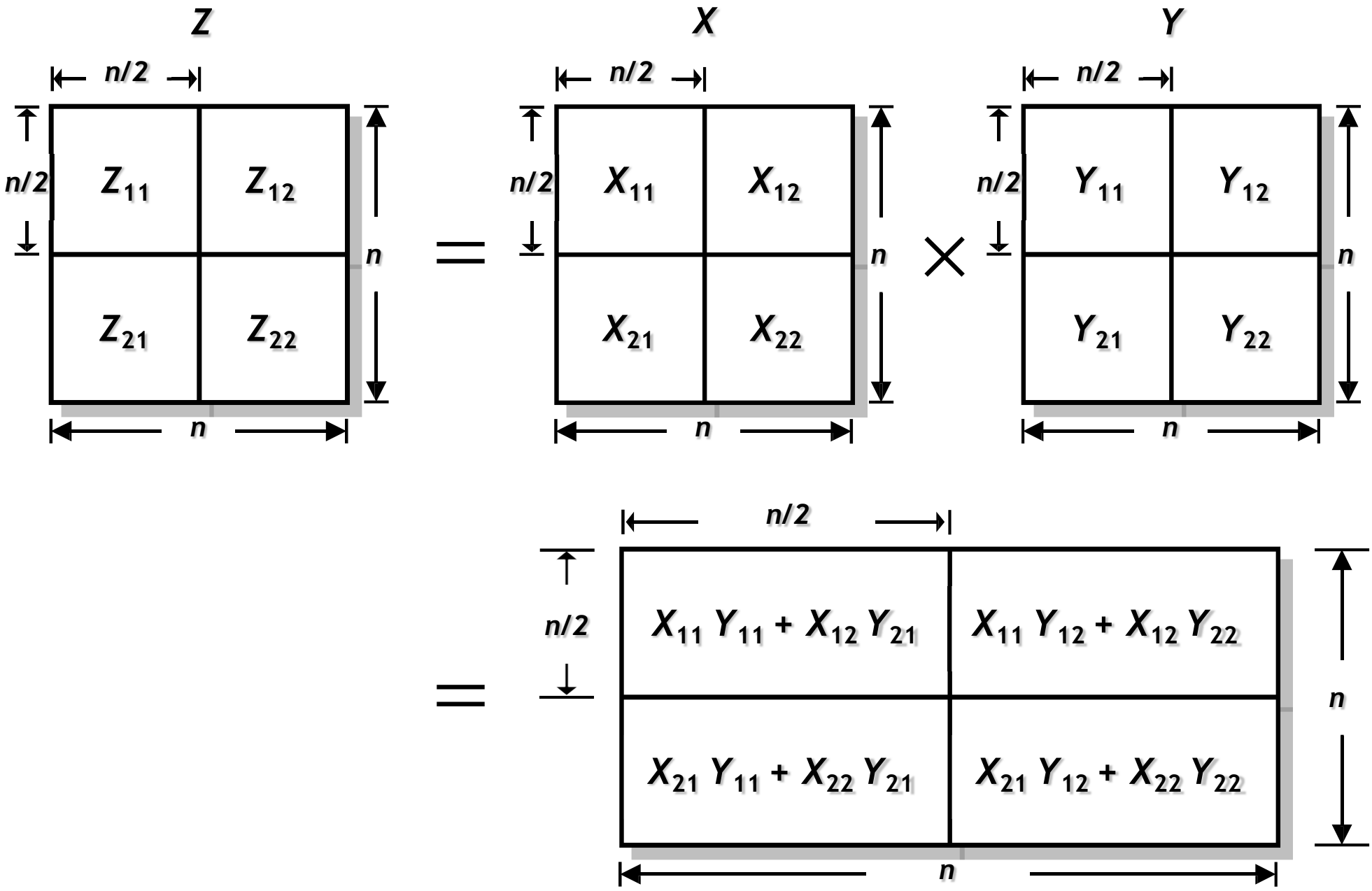
Multiple Levels of Cache



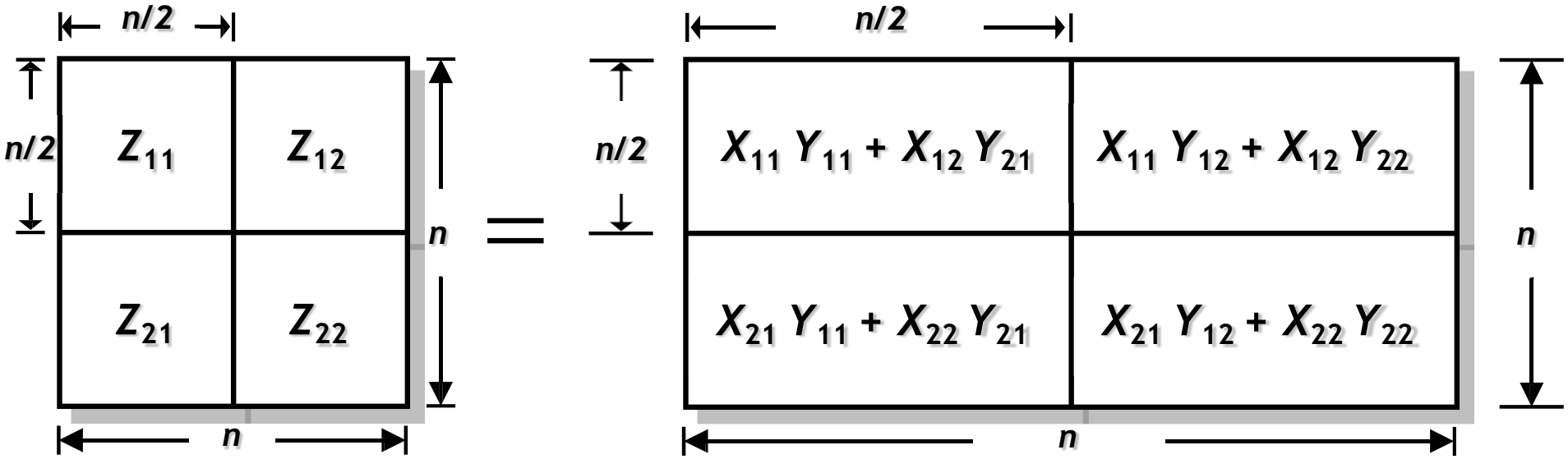
One Parameter Per Caching Level!

```
Block-MM( X, Y, Z, n )
1.  for i1 ← 1 to n / s do
2.      for j1 ← 1 to n / s do
3.          for k1 ← 1 to n / s do
4.              for i2 ← 1 to s / t do
5.                  for j2 ← 1 to s / t do
6.                      for k2 ← 1 to s / t do
7.                          Iter-MM( (X_{i1k1})_{i2k2}, (Y_{k1j1})_{k2j2}, (X_{i1j1})_{i2j2}, t )
```

Recursive Matrix Multiplication



Recursive Matrix Multiplication



Rec-MM(Z, X, Y)

1. *if* $Z \equiv 1 \times 1$ matrix *then* $Z \leftarrow Z + X \cdot Y$
2. *else*
3. *Rec-MM* (Z_{11}, X_{11}, Y_{11}), *Rec-MM* (Z_{11}, X_{12}, Y_{21})
4. *Rec-MM* (Z_{12}, X_{12}, Y_{12}), *Rec-MM* (Z_{12}, X_{12}, Y_{22})
5. *Rec-MM* (Z_{21}, X_{21}, Y_{11}), *Rec-MM* (Z_{21}, X_{22}, Y_{21})
6. *Rec-MM* (Z_{22}, X_{21}, Y_{12}), *Rec-MM* (Z_{22}, X_{22}, Y_{22})

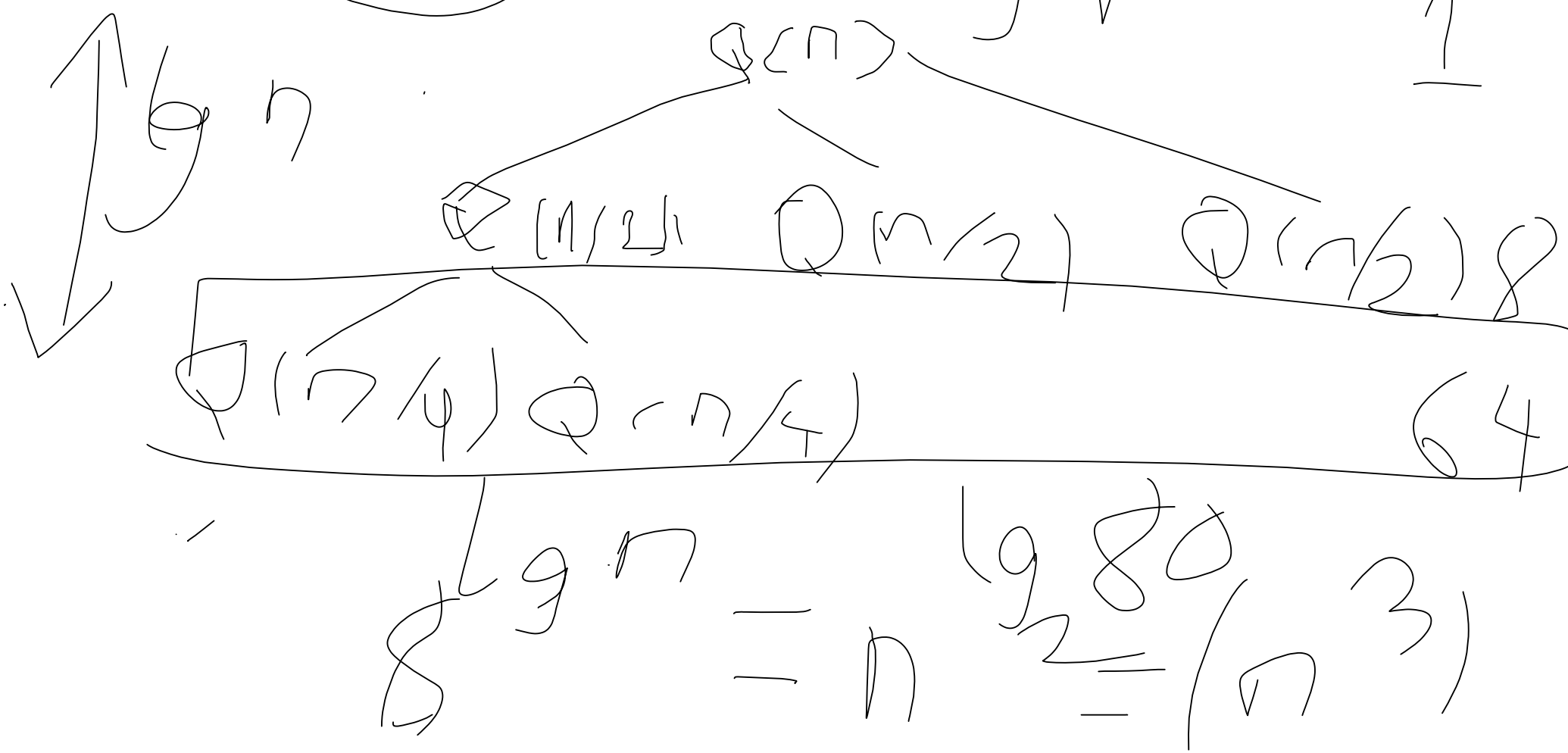
$2^k = n$

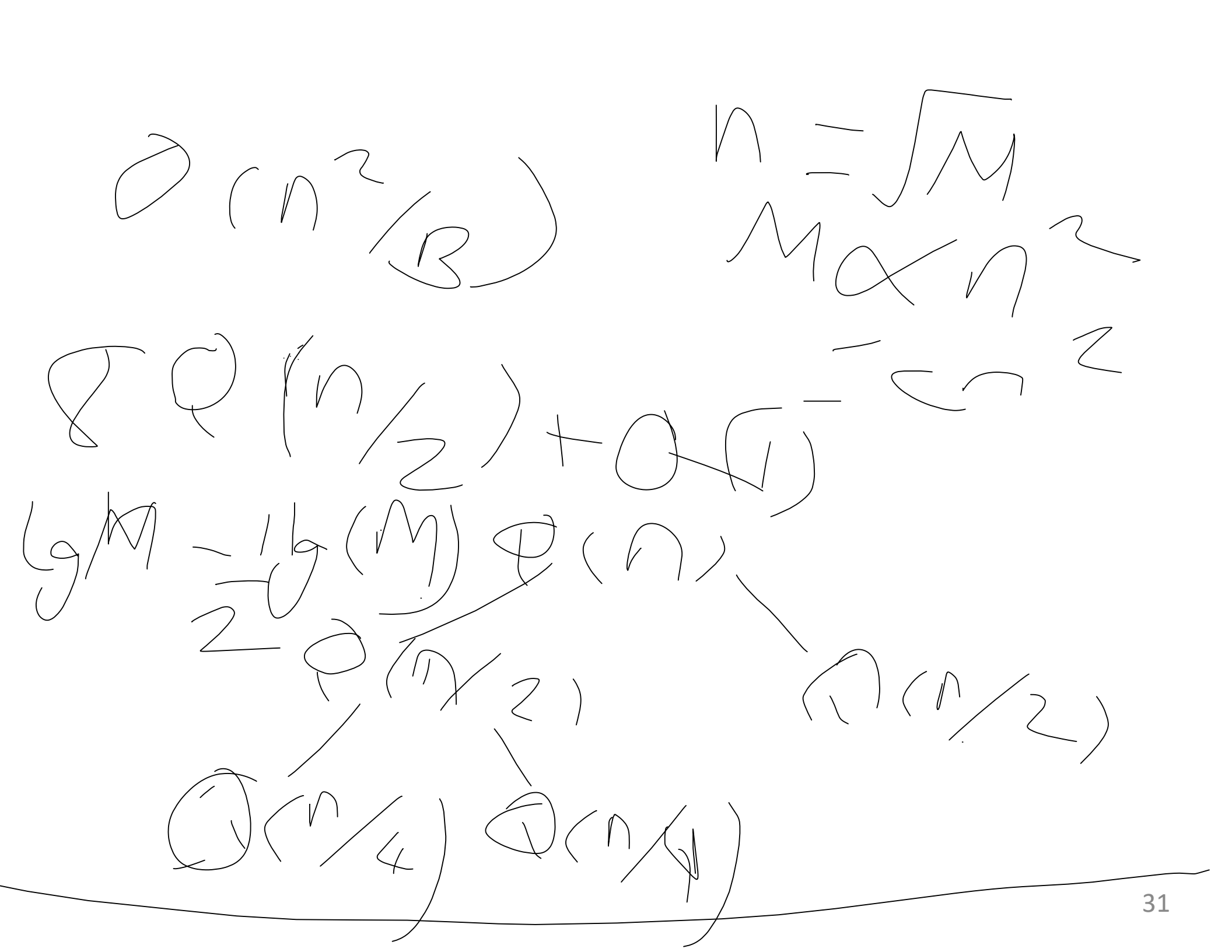
$k = \log n$

16, 8, 4, 2, 1

$k = \log n$

$Q(n) = 8Q(n/2)+O(1)$




O(n^2/B)
n = √M
M ∝ n^2
Q(n/2) + O(1)
O(M/B) Q(n)
2 Q(n/2)
O(n/2)
O(n/4) O(n/4)

$$n_2^h = \sqrt{cm}$$

$$\log(n_2^h) = \log(\sqrt{cm})$$

$$\log n - \log 2^h = \log(\sqrt{cm})$$

$$\log n - h = \log(\sqrt{cm})$$

$$h = \log n / \log(\sqrt{cm})$$

Leaves

$$\lg n - \frac{1}{2}\lg(cM)$$

$$= \frac{n^3}{M^{3/2}} \times M/B$$

$$= \frac{n^3}{\sqrt{M}B}$$

Recursive Matrix Multiplication

Rec-MM (Z, X, Y)

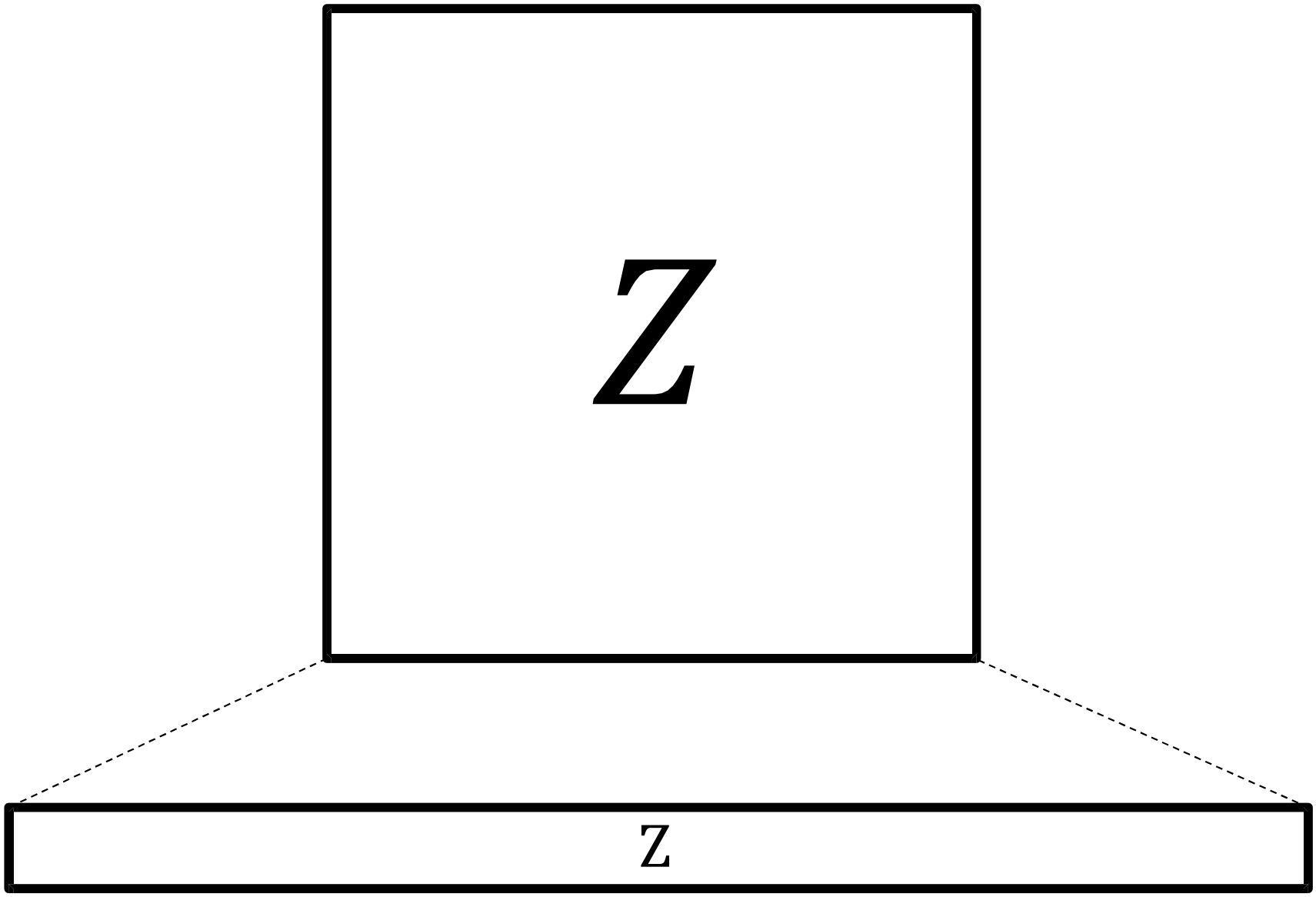
1. *if* $Z \equiv 1 \times 1$ matrix *then* $Z \leftarrow Z + X \cdot Y$
2. *else*
3. *Rec-MM* (Z_{11}, X_{11}, Y_{11}), *Rec-MM* (Z_{11}, X_{12}, Y_{21})
4. *Rec-MM* (Z_{12}, X_{12}, Y_{12}), *Rec-MM* (Z_{12}, X_{12}, Y_{22})
5. *Rec-MM* (Z_{21}, X_{21}, Y_{11}), *Rec-MM* (Z_{21}, X_{22}, Y_{21})
6. *Rec-MM* (Z_{22}, X_{21}, Y_{12}), *Rec-MM* (Z_{22}, X_{22}, Y_{22})

I/O-complexity (for $n > M$), $Q(n) = \begin{cases} O\left(n + \frac{n^2}{B}\right), & \text{if } n^2 \leq \alpha M \\ 8Q\left(\frac{n}{2}\right) + O(1), & \text{otherwise} \end{cases}$

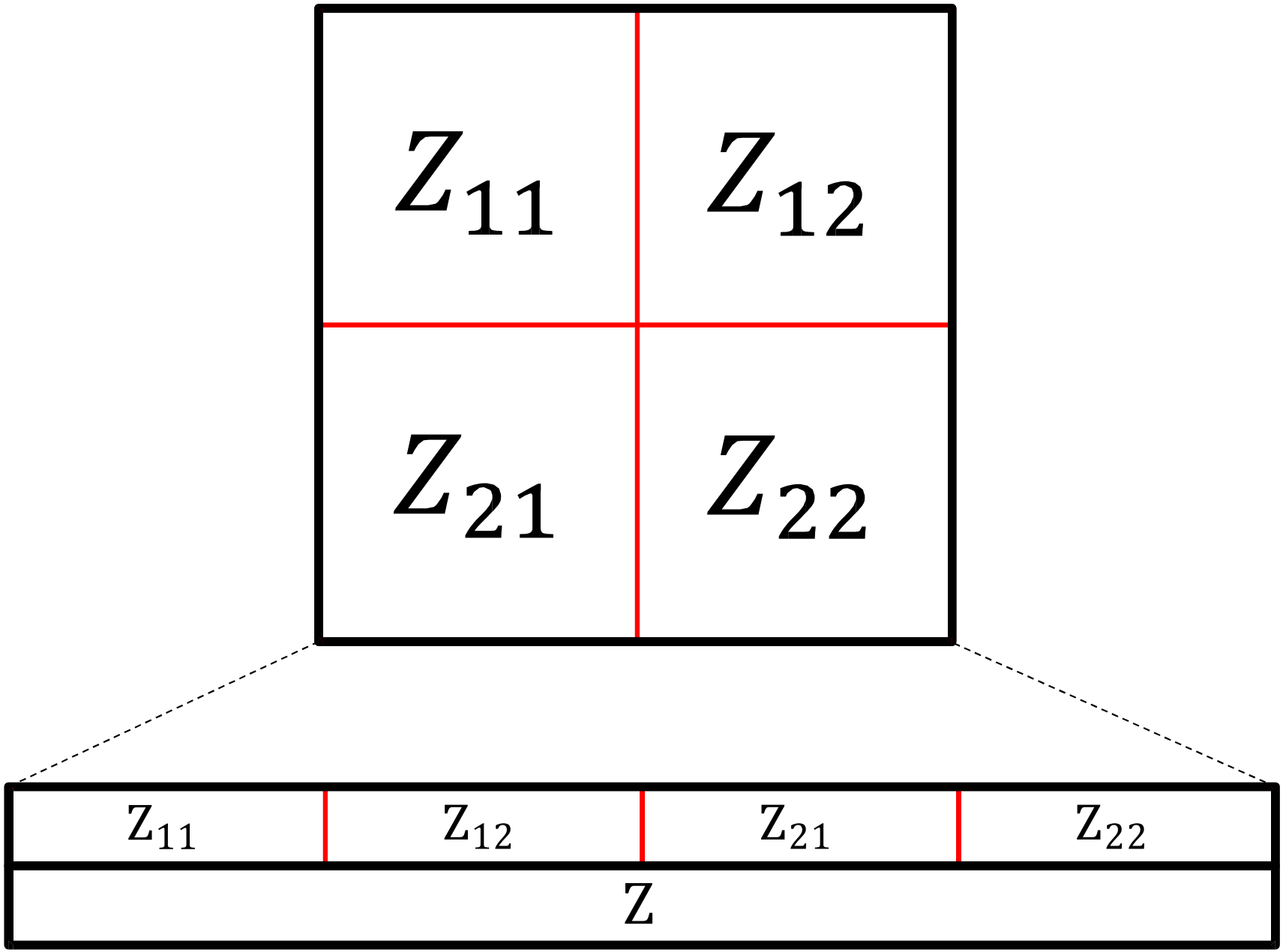
$$= O\left(\frac{n^3}{M} + \frac{n^3}{B\sqrt{M}}\right) = O\left(\frac{n^3}{B\sqrt{M}}\right), \text{when } M = \Omega(B^2)$$

$$\text{I/O-complexity (for all } n \text{)} = O\left(\frac{n^3}{B\sqrt{M}} + \frac{n^2}{B} + 1\right)$$

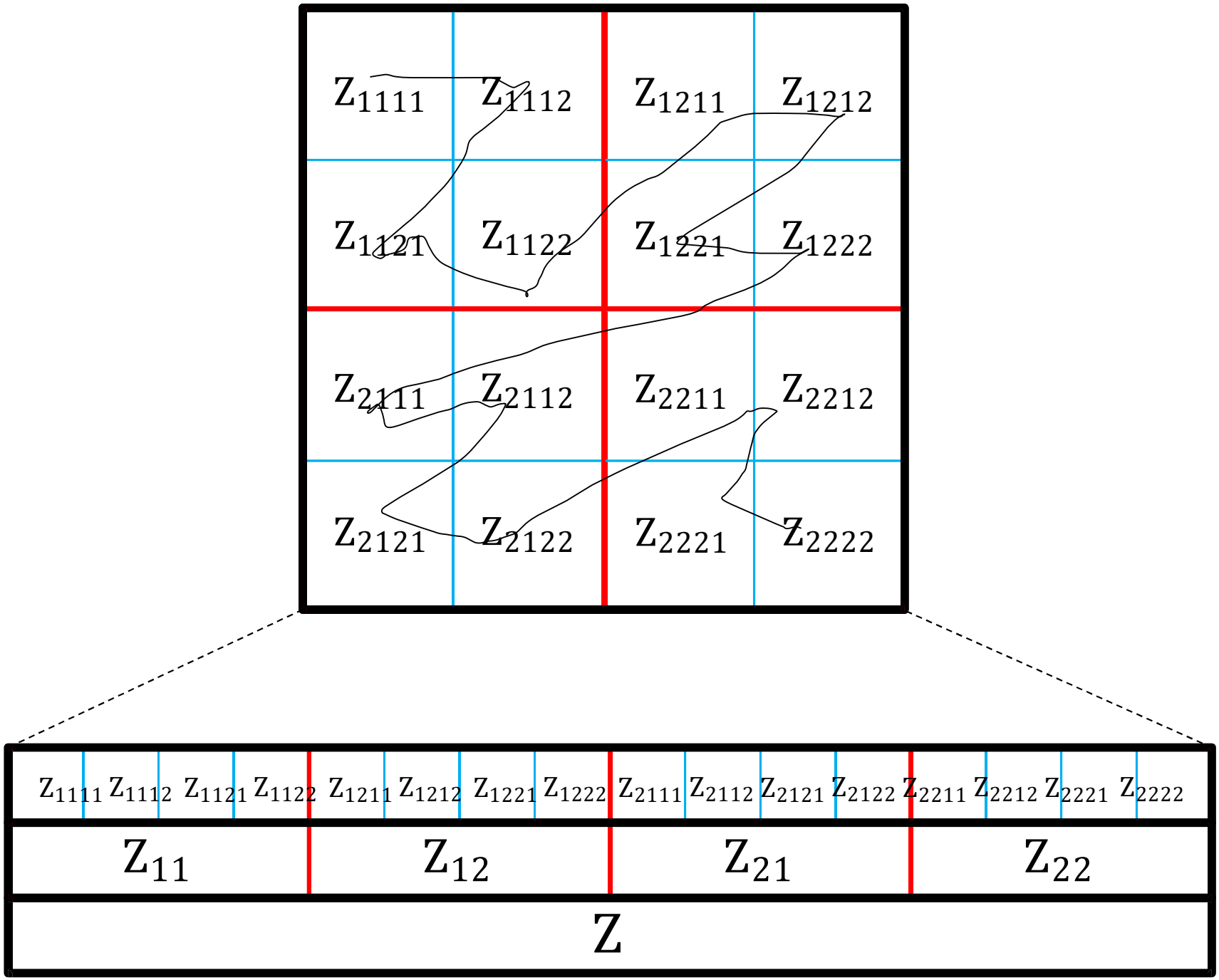
Recursive Matrix Multiplication with Z-Morton Layout



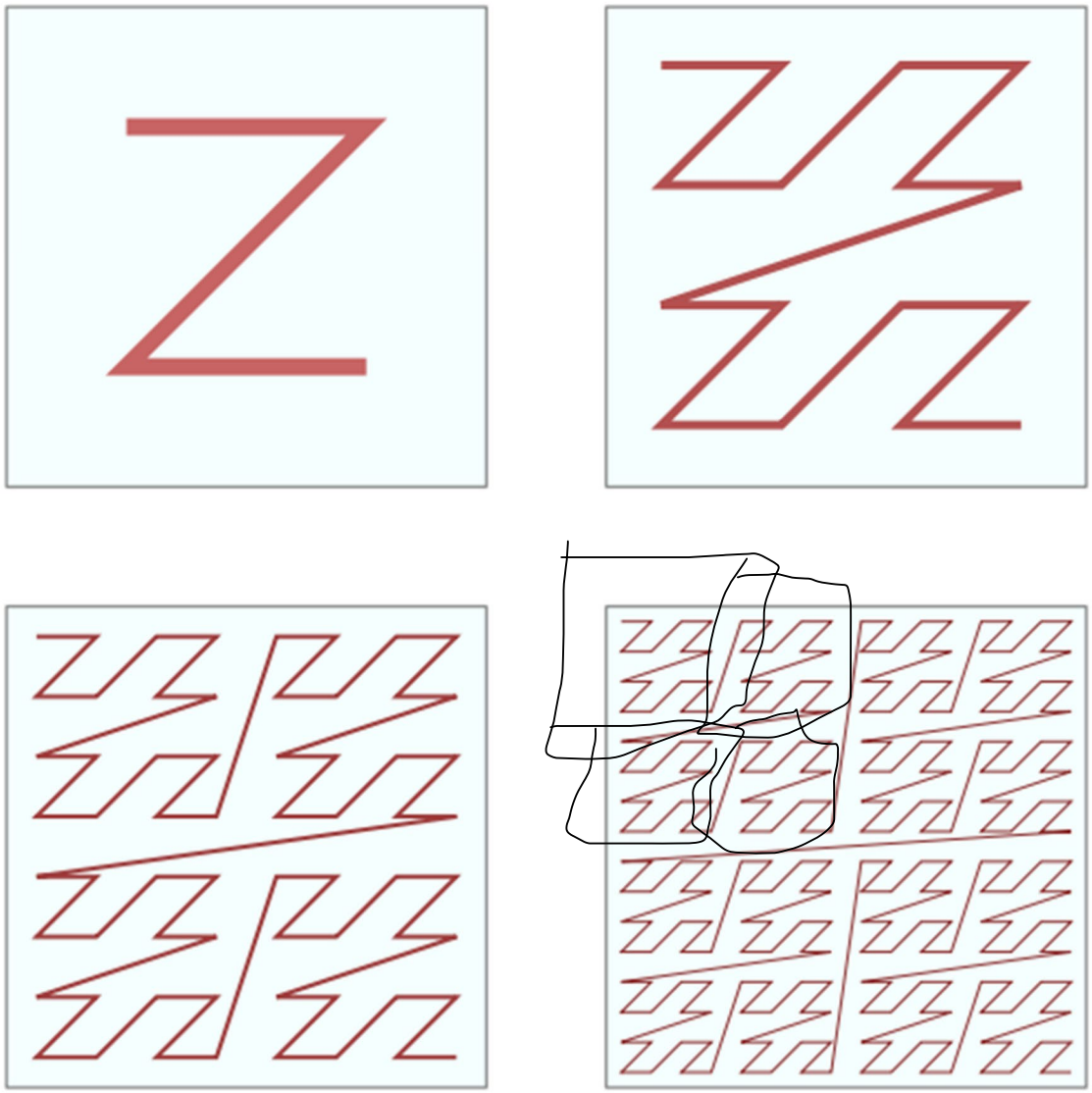
Recursive Matrix Multiplication with Z-Morton Layout



Recursive Matrix Multiplication with Z-Morton Layout



Recursive Matrix Multiplication with Z-Morton Layout



Source: wikipedia

Recursive Matrix Multiplication with Z-Morton Layout

Rec-MM (Z, X, Y)

1. *if* $Z \equiv 1 \times 1$ matrix *then* $Z \leftarrow Z + X \cdot Y$
2. *else*
3. *Rec-MM* (Z_{11}, X_{11}, Y_{11}), *Rec-MM* (Z_{11}, X_{12}, Y_{21})
4. *Rec-MM* (Z_{12}, X_{12}, Y_{12}), *Rec-MM* (Z_{12}, X_{12}, Y_{22})
5. *Rec-MM* (Z_{21}, X_{21}, Y_{11}), *Rec-MM* (Z_{21}, X_{22}, Y_{21})
6. *Rec-MM* (Z_{22}, X_{21}, Y_{12}), *Rec-MM* (Z_{22}, X_{22}, Y_{22})

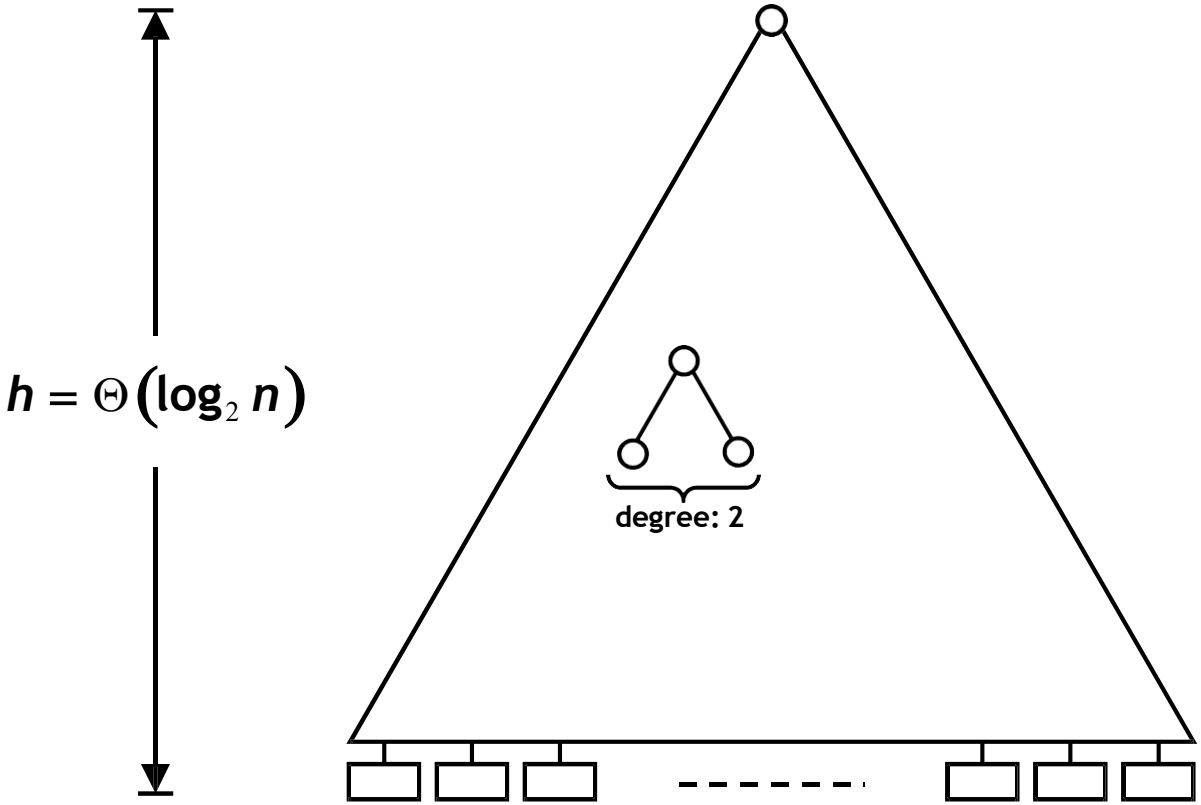
I/O-complexity (for $n \Sigma M$), $Q(n) = \begin{cases} O\left(1 + \frac{n^2}{B}\right), & \text{if } n^2 \leq \alpha M \\ 8Q\left(\frac{n}{2}\right) + O(1) & \text{otherwise} \end{cases}$

$$= O\left(\frac{n^3}{M\sqrt{M}} + \frac{n^3}{B\sqrt{M}}\right) = O\left(\frac{n^3}{B\sqrt{M}}\right), \text{when } M = \Omega(B)$$

I/O-complexity (for all n) $= O\left(\frac{n^3}{B\sqrt{M}} + \frac{n^2}{B} + 1\right)$

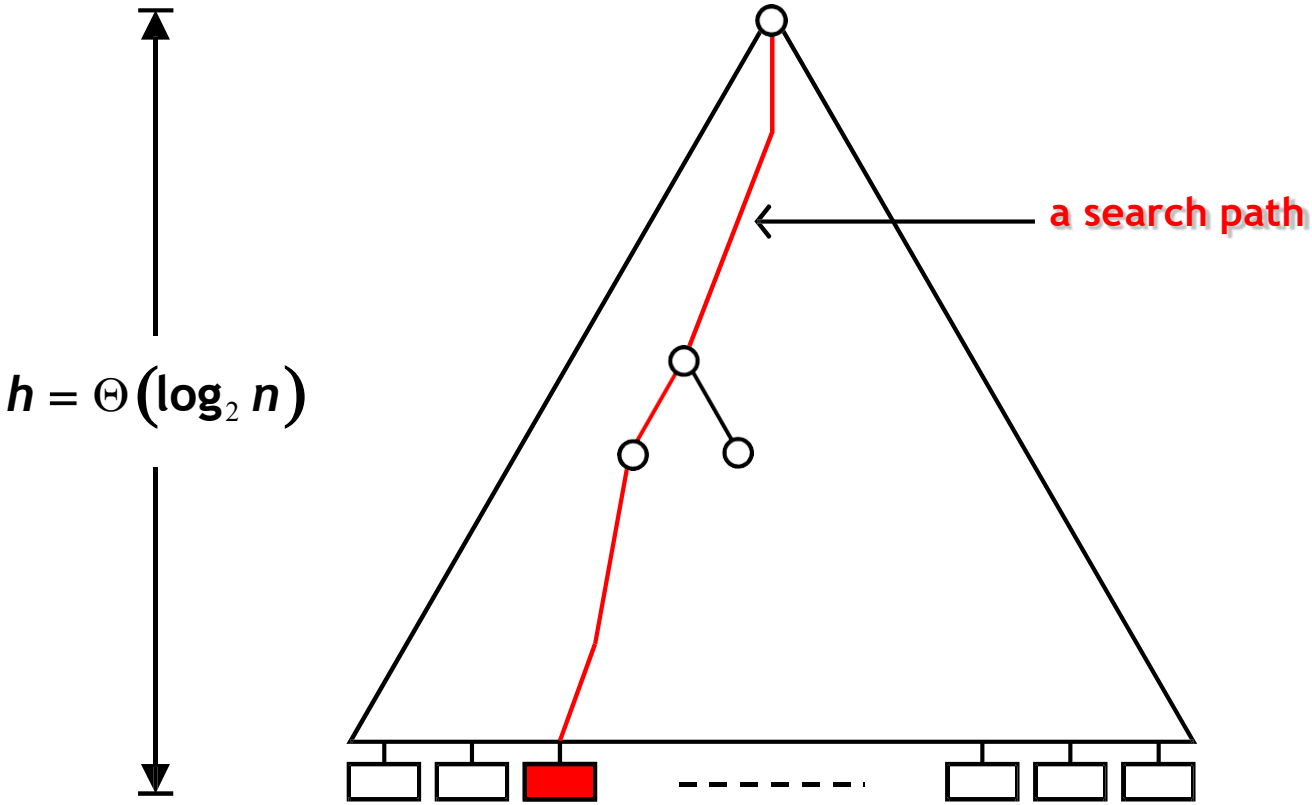
Searching
(Static B-Trees)

A Static Search Tree



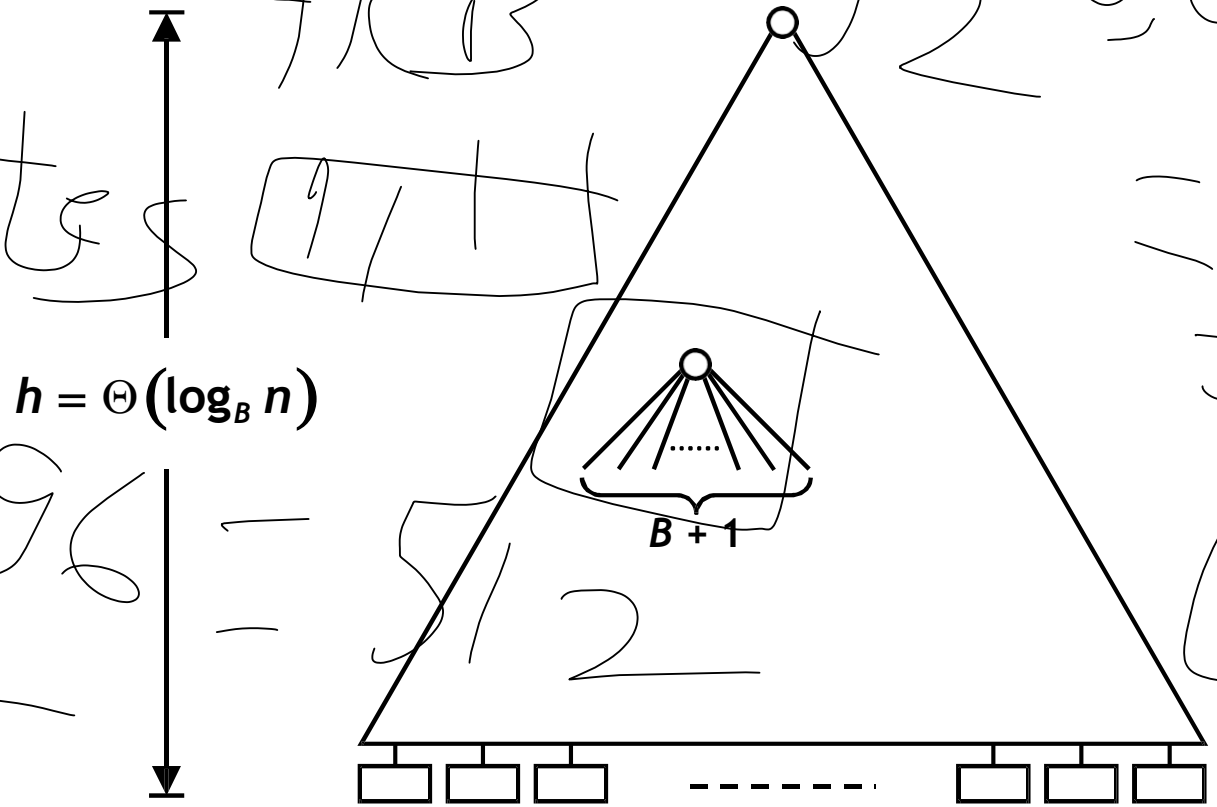
- A perfectly balanced binary search tree
- Static: no insertions or deletions
- Height of the tree, $h = \Theta(\log_2 n)$

A Static Search Tree



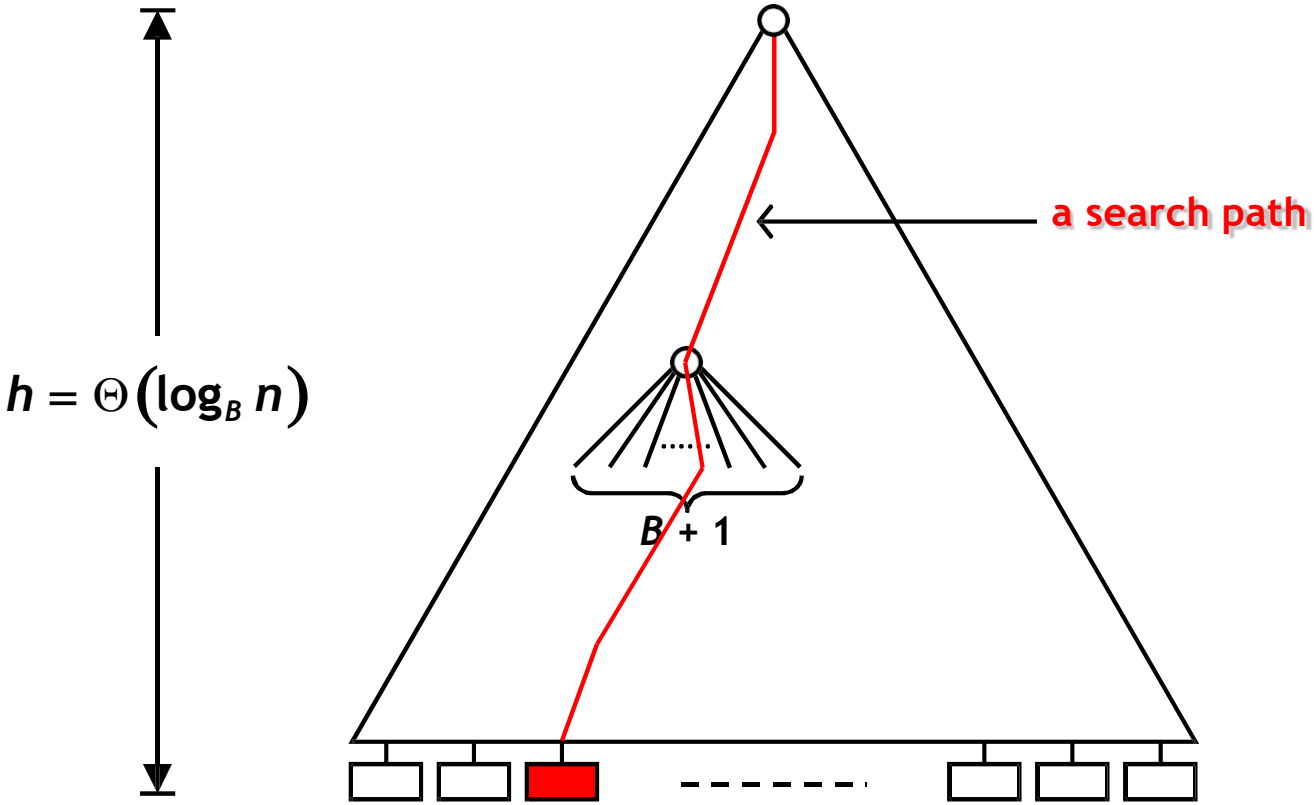
- A perfectly balanced binary search tree
- Static: no insertions or deletions
- Height of the tree, $h = \Theta(\log_2 n)$
- A search path visits $O(h)$ nodes, and incurs $O(h) = O(\log_2 n)$ I/Os

I/O-Efficient Static B-Trees



- Each node stores B keys, and has degree $B + 1$
- Height of the tree, $h = \Theta(\log_B n)$

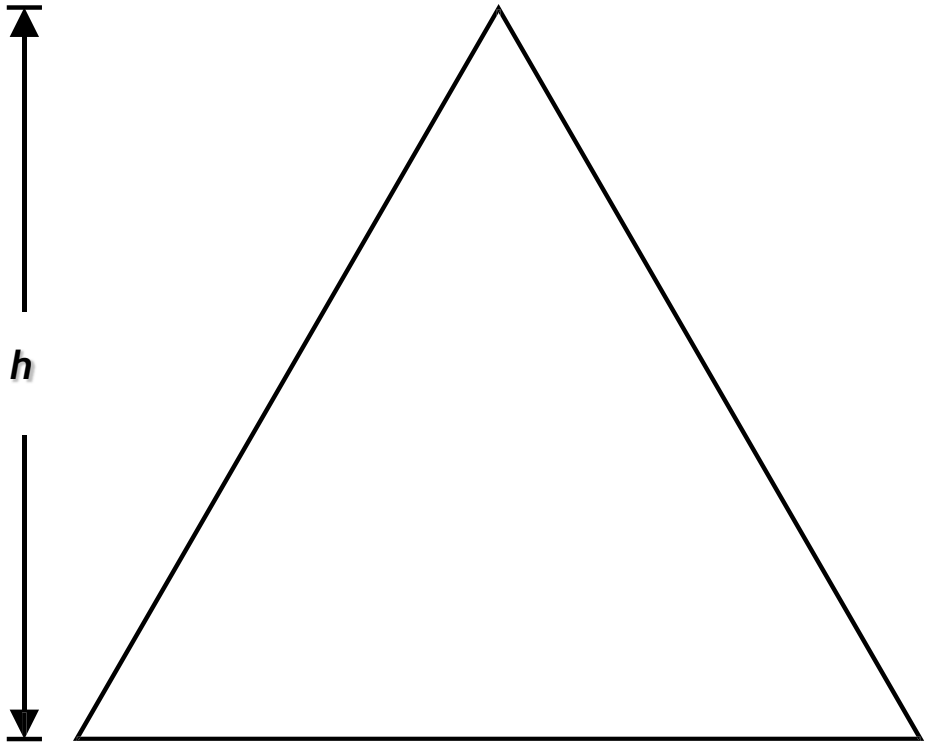
I/O-Efficient Static B-Trees



- Each node stores B keys, and has degree $B + 1$
- Height of the tree, $h = \Theta(\log_B n)$
- A search path visits $O(h)$ nodes, and incurs $O(h) = O(\log_B n)$ I/Os

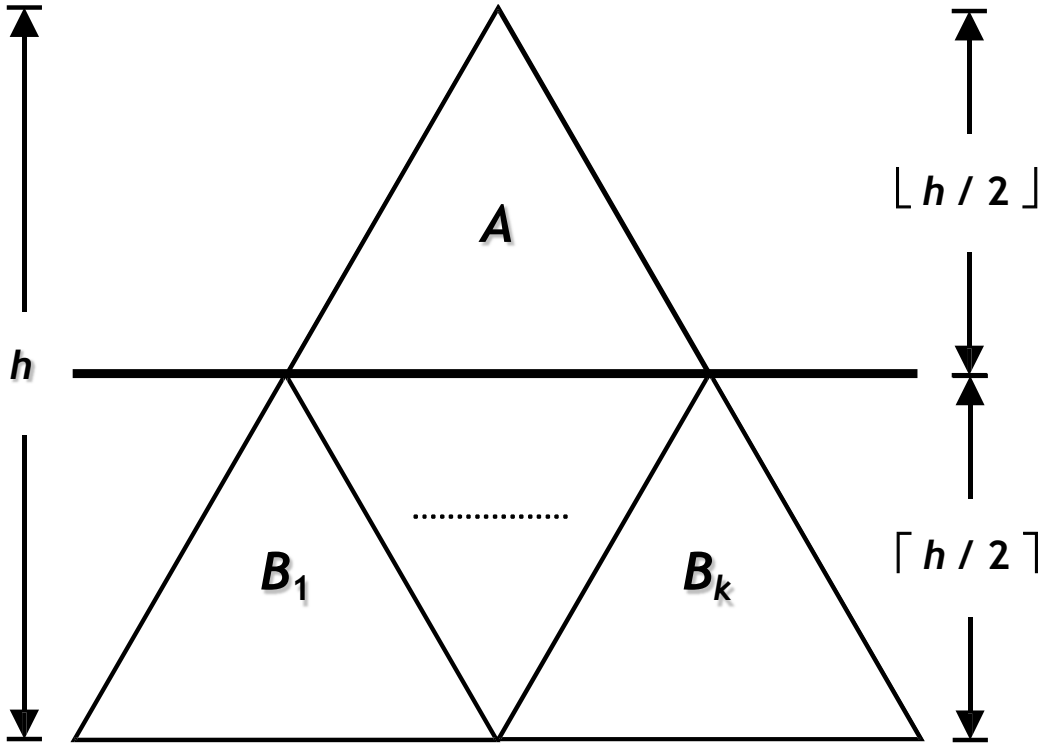
Cache-Oblivious Static B-Trees?

van Emde Boas Layout



a binary search tree

van Emde Boas Layout



a binary search tree

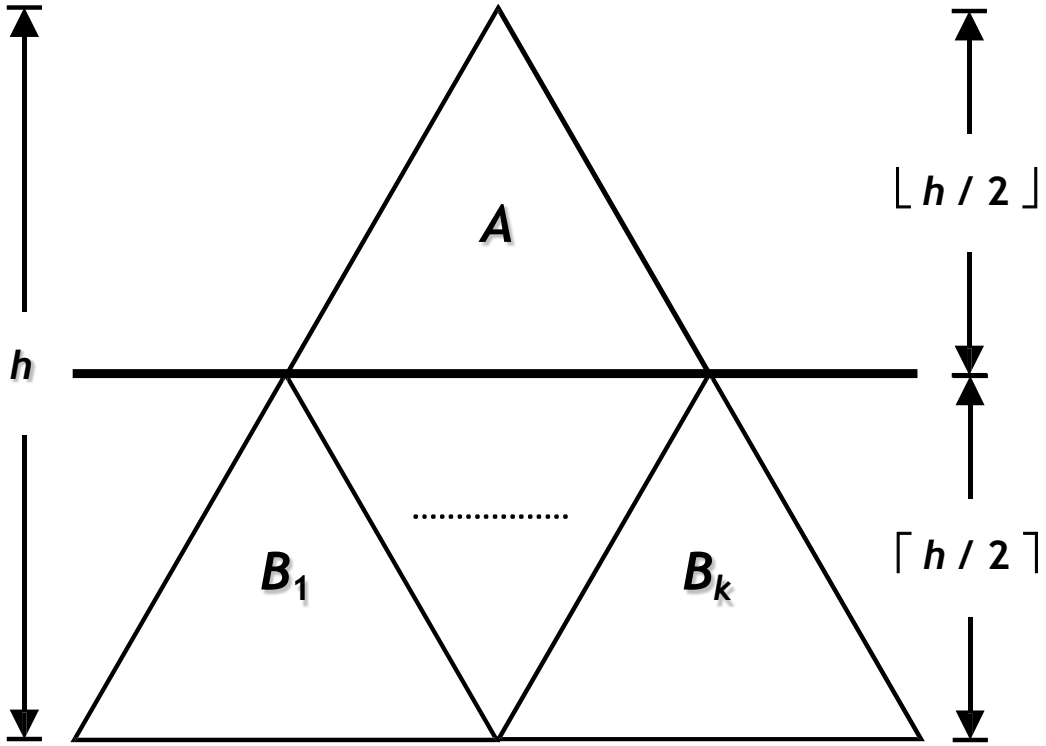


If the tree contains n nodes,

each subtree contains $\Theta\left(2^{h/2}\right) = \Theta(\sqrt{n})$ nodes,

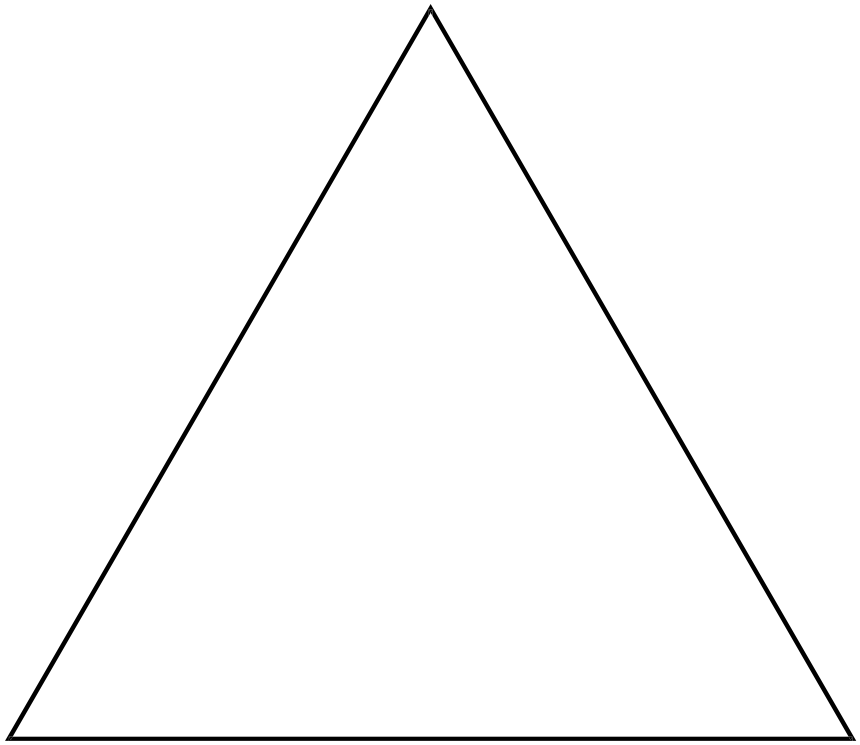
and $k = \Theta(\sqrt{n})$.

van Emde Boas Layout



a binary search tree





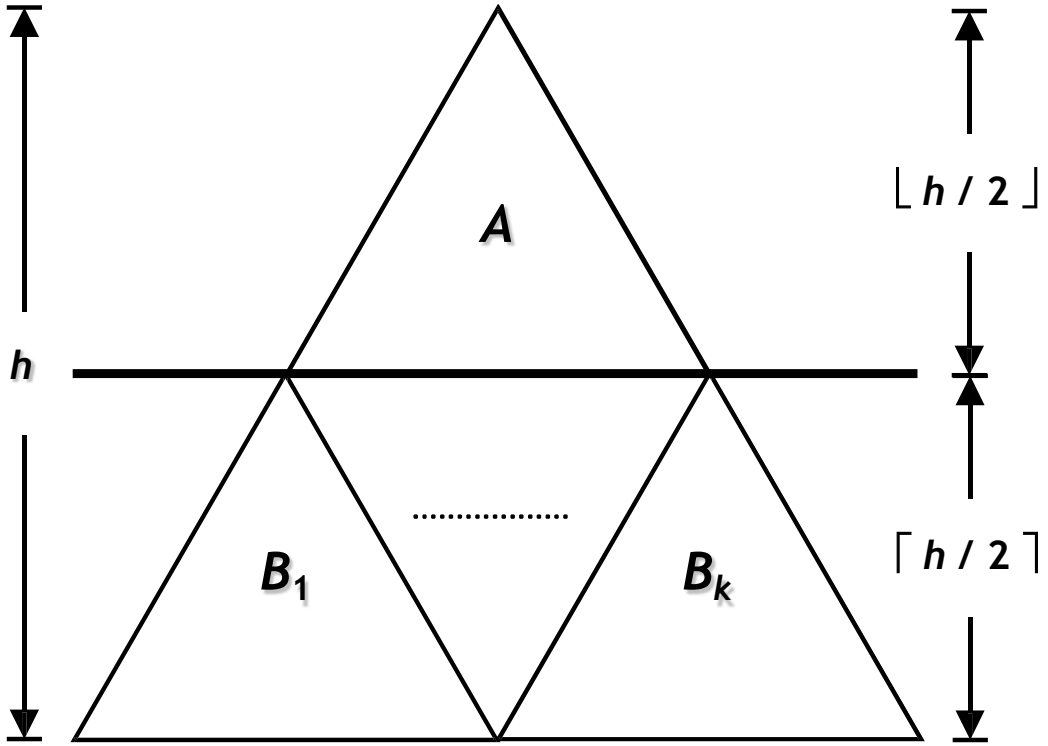
Recursive Subdivision

If the tree contains n nodes,

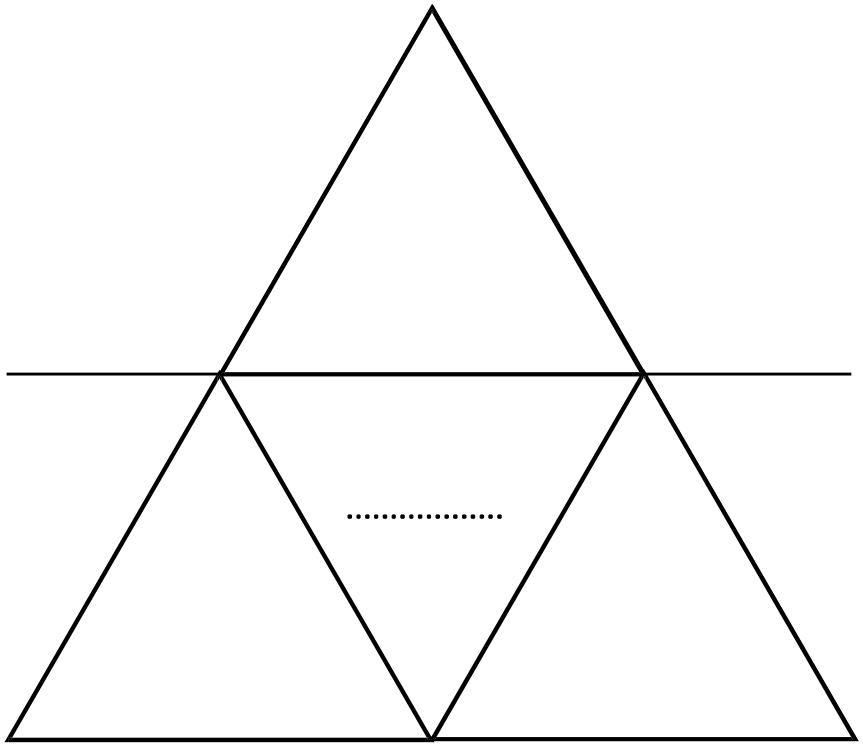
each subtree contains $\Theta\left(2^{h/2}\right) = \Theta(\sqrt{n})$ nodes,

and $k = \Theta(\sqrt{n})$.

van Emde Boas Layout



a binary search tree



Recursive Subdivision

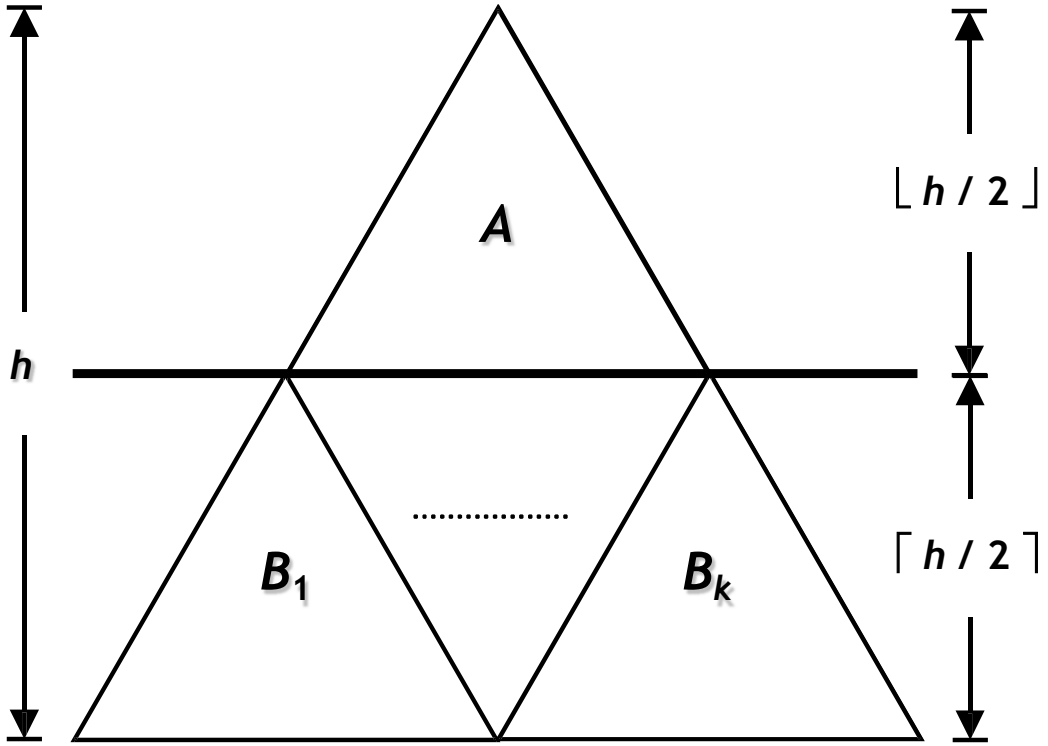
A	B_1	B_2	.....	B_k

If the tree contains n nodes,

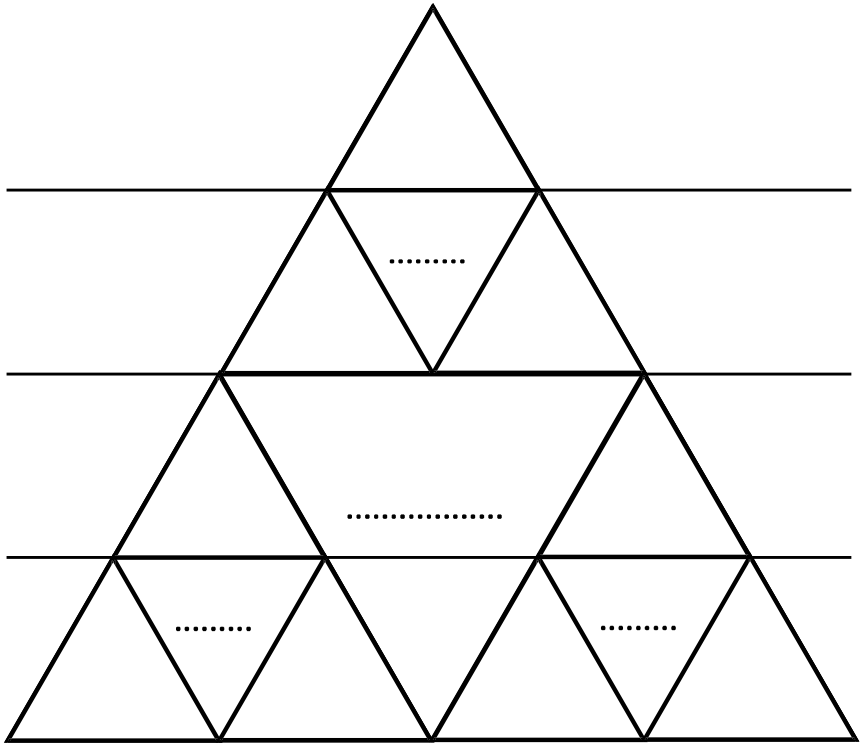
each subtree contains $\Theta\left(2^{h/2}\right) = \Theta(\sqrt{n})$ nodes,

and $k = \Theta(\sqrt{n})$.

van Emde Boas Layout



a binary search tree



Recursive Subdivision

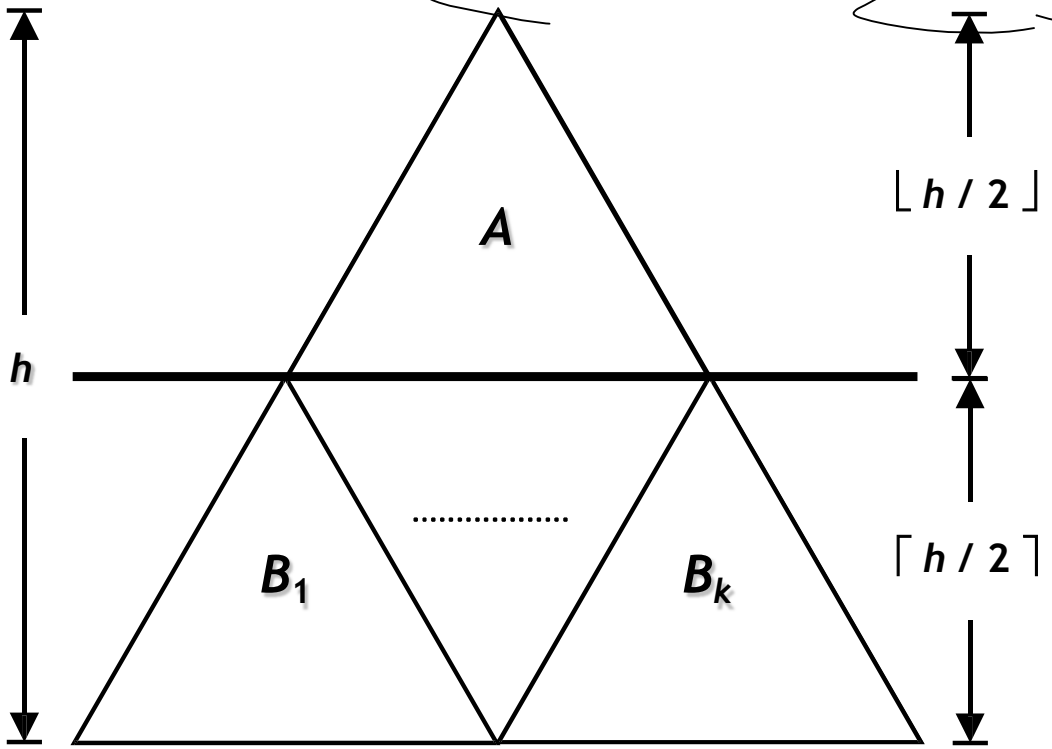
A	B_1	B_2	.....	B_k

If the tree contains n nodes,

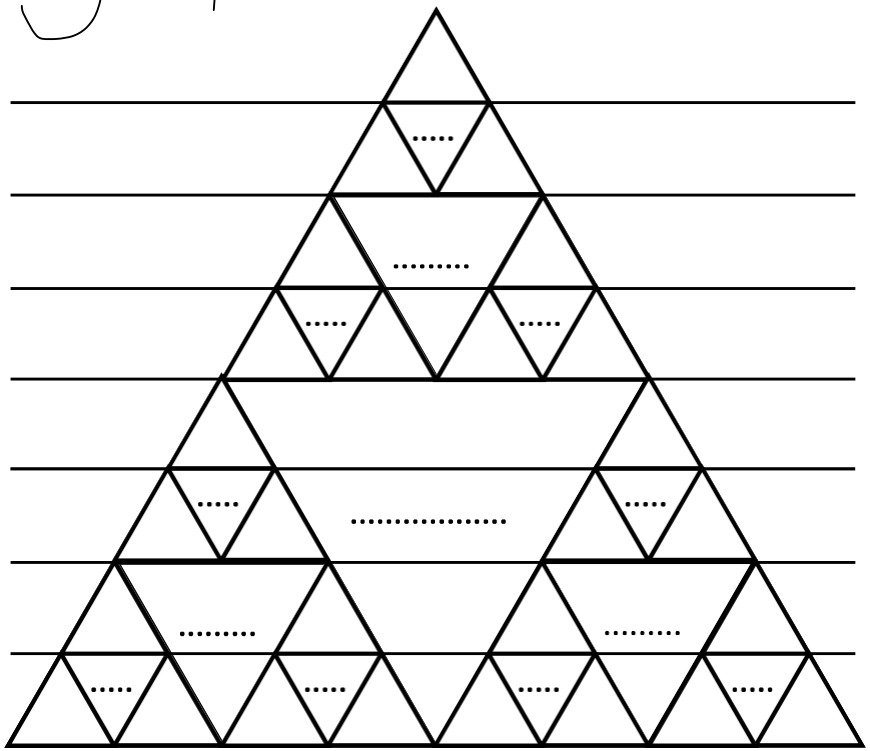
each subtree contains $\Theta\left(2^{h/2}\right) = \Theta(\sqrt{n})$ nodes,

and $k = \Theta(\sqrt{n})$.

van Emde Boas Layout



a binary search tree



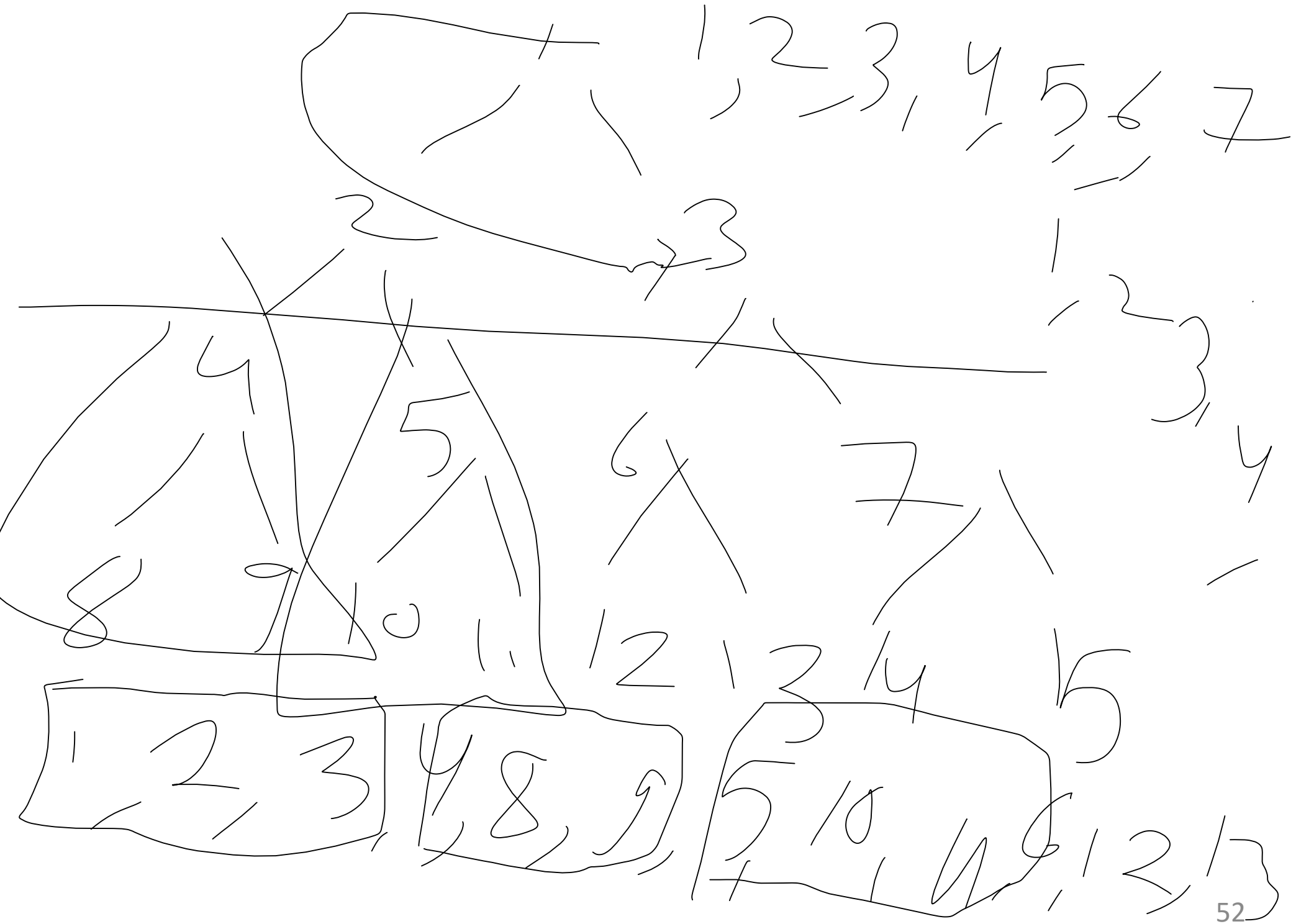
Recursive Subdivision

A	B_1	B_2	.....	B_k

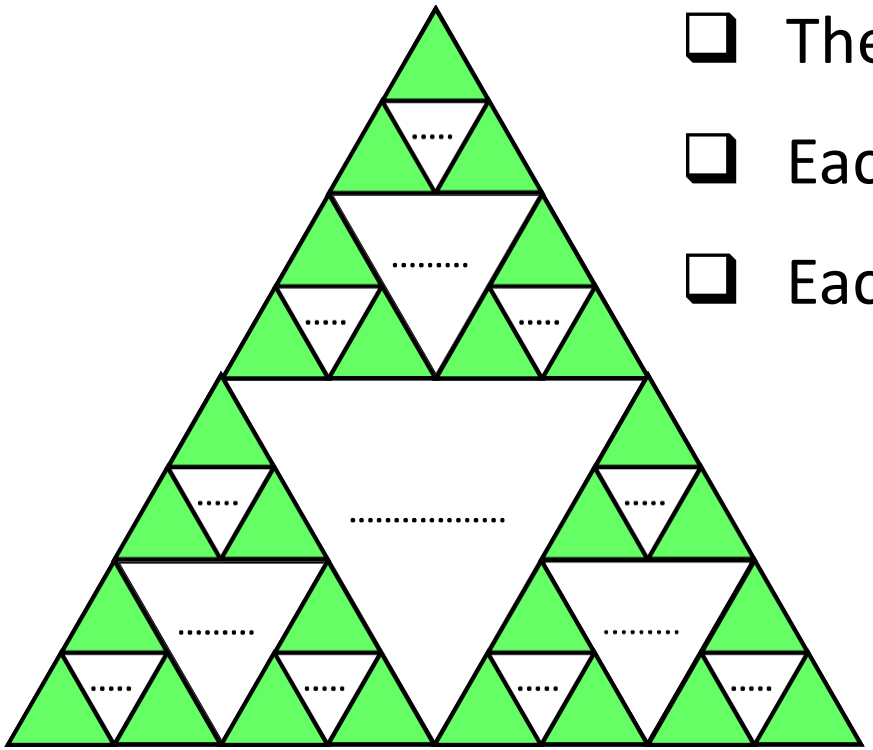
If the tree contains n nodes,

each subtree contains $\Theta\left(2^{h/2}\right) = \Theta(\sqrt{n})$ nodes,

and $k = \Theta(\sqrt{n})$.

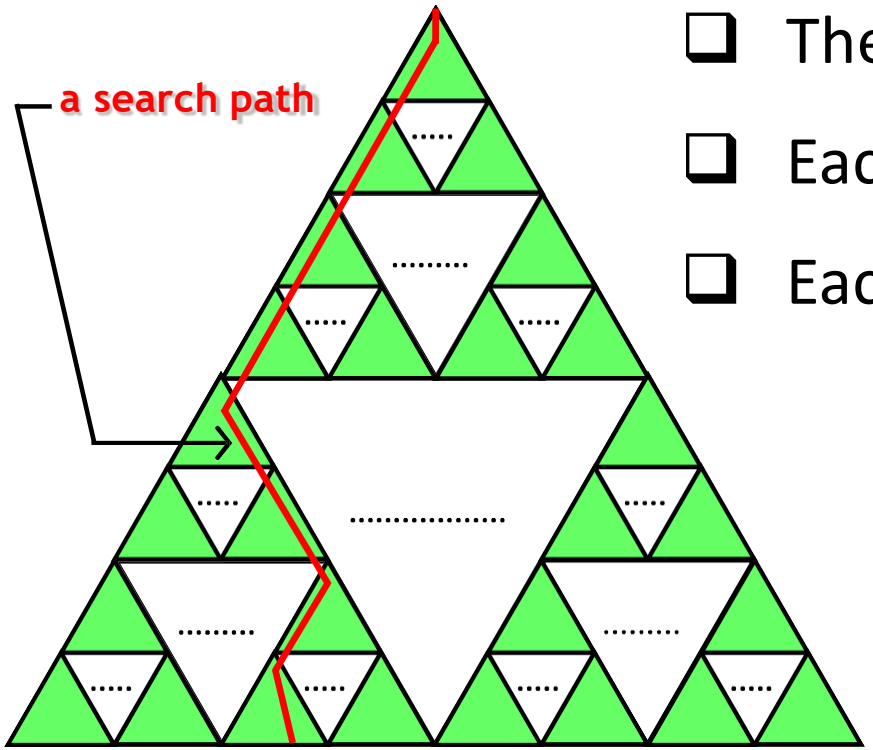


I/O-Complexity of a Search



- The height of the tree is log n
- Each △ has height between $\frac{1}{2}\log B$ & $\log B$.
- Each △ spans at most 2 blocks of size B.

I/O-Complexity of a Search



- The height of the tree is log n
- Each △ has height between $\frac{1}{2}\log B$ & $\log B$.
- Each △ spans at most 2 blocks of size B.

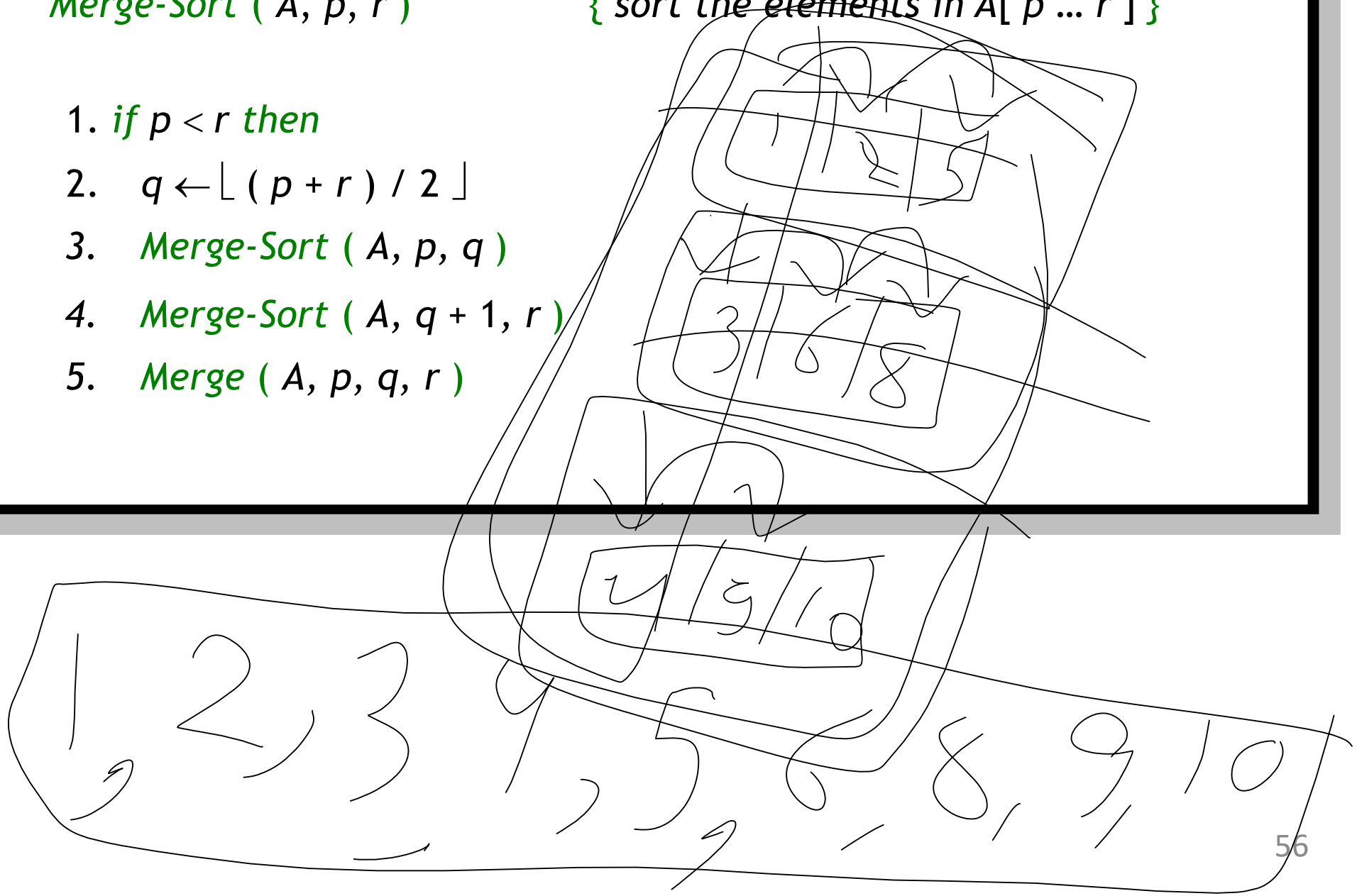
- p = number of △'s visited by a search path
- Then $p \leq \frac{\log n}{\log B} = \log_B n$, and $p \leq \frac{\log n}{\frac{1}{2}\log B} = 2\log_B n$
- The number of blocks transferred is $\leq 2 \times 2 \log_B n = 4 \log_B n$

Sorting
(Mergesort)

Merge Sort

```
Merge-Sort ( A, p, r )          { sort the elements in A[ p ... r ] }

1. if p < r then
2.     q ← ⌊ ( p + r ) / 2 ⌋
3.     Merge-Sort ( A, p, q )
4.     Merge-Sort ( A, q + 1, r )
5.     Merge ( A, p, q, r )
```



Merging *k* Sorted Sequences

- $k \geq 2$ sorted sequences $S_1, S_2, \ldots, S_k$ stored in external memory
- $|S_i| = n_i$ for $1 \leq i \leq k$
- $n = n_1 + n_2 + \cdots + n_k$ is the length of the merged sequence S
- S (initially empty) will be stored in external memory
- Cache must be large enough to store
  - one block from each S_i
  - one block from S

  Thus $M \geq (k + 1)B$

Merging *k* Sorted Sequences

- Let $\mathcal{B}_i$ be the cache block associated with S_i, and let $\mathcal{B}$ bethe block associated with S (initially all empty)
- Whenever a $\mathcal{B}_i$ is empty fill it up with the next block from S_i
- Keep transferring the next smallest element among all $\mathcal{B}_i$s to $\mathcal{B}$
- Whenever $\mathcal{B}$ becomes full, empty it by appending it to S
- In the *Ideal Cache Model* the block emptying and replacements will happen automatically $\Rightarrow$ cache-oblivious merging

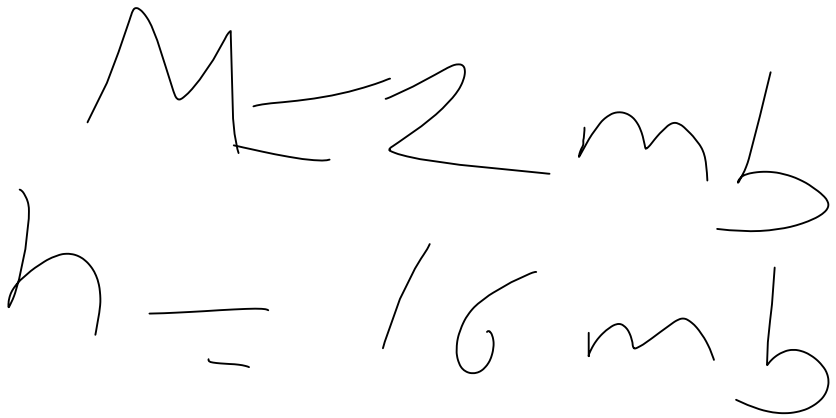
I/O Complexity

- Reading S_i: #block transfers $\leq 2 + \frac{n_i}{B}$
- Writing S: #block transfers $\leq 1 + \frac{n}{B}$
- Total #block transfers $\leq 1 + \frac{n}{B} + \sum_{1 \text{ to } k} \left(2 + \frac{n_i}{B}\right) = O\left(k + \frac{n}{B}\right)$

Cache-Oblivious 2-Way Merge Sort

Merge-Sort (*A*, *p*, *r*) { *sort the elements in A*[*p* … *r*] }

1. *if* $p < r$ *then*
2. $q \leftarrow \lfloor (p + r) / 2 \rfloor$
3. *Merge-Sort* (*A*, *p*, *q*)
4. *Merge-Sort* (*A*, *q* + 1, *r*)
5. *Merge* (*A*, *p*, *q*, *r*)



I/O Complexity: O(n/B log n/M)

Cache-Oblivious *k*-Way Merge Sort

I/O Complexity: $Q(n)$ = $O(k\ n/M + n/B \log_k n/M)$

How large can k be?

Recall that for k-way merging, we must ensure

$$M \geq (k+1)B \Rightarrow k \leq \frac{M}{B} - 1$$

Cache-Aware $\left(\frac{M}{B}-1\right)$-Way Merge Sort

I/O Complexity: $Q(n) = \begin{cases} O\left(1+\frac{n}{B}\right), & \text{if } n \le M, \\ k \cdot Q\left(\frac{n}{k}\right) + O\left(k+\frac{n}{B}\right), & \text{otherwise.} \end{cases}$

$$= O\left(k \cdot \frac{n}{M} + \frac{n}{B}\log_k \frac{n}{M}\right)$$

Using $k = \frac{M}{B}-1$, we get:

$$Q(n) = O\left(\left(\frac{M}{B}-1\right)\frac{n}{M} + \frac{n}{B}\log_{\frac{M}{B}}\left(\frac{n}{M}\right)\right) = O\left(\frac{n}{B}\log_{\frac{M}{B}}\left(\frac{n}{M}\right)\right)$$

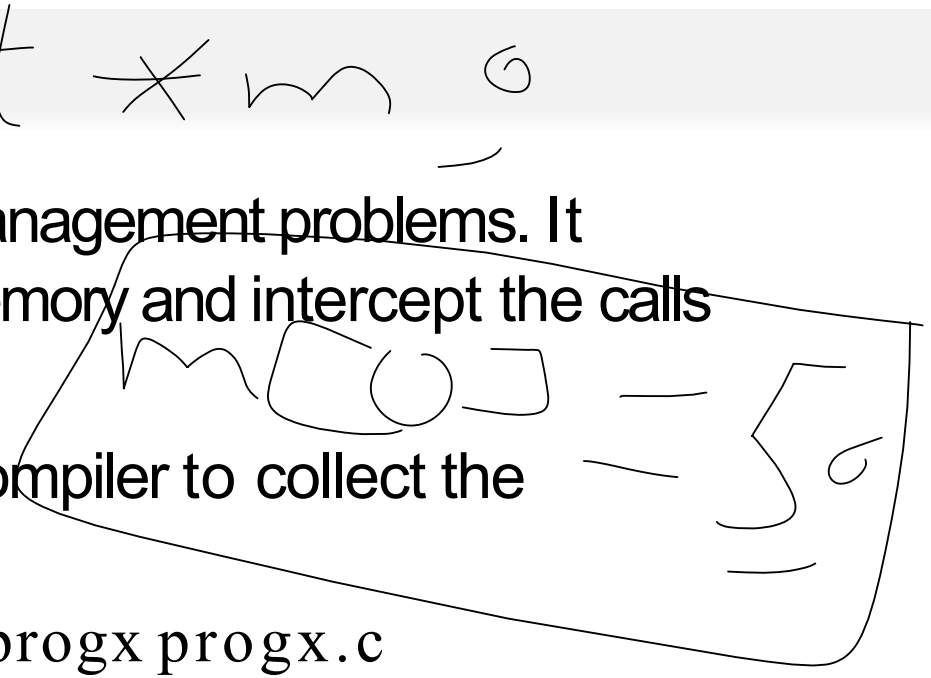
Valgrind

Valgrind is a framework that includes tools for automatically detecting many memory management and threading bugs and profiling C/C++ programs. The current Valgrind distribution includes the following production-quality tools:

1. A memory error detector
2. Two thread error detectors
3. A cache and branch-prediction profiler
4. A call-graph generating cache
5. A branch-prediction profiler
6. A heap profiler.

It also includes three experimental tools: a stack/global array overrun detector, a second heap profiler that examines how heap blocks are used, and a SimPoint basic block vector generator.

Memcheck



The *Memcheck* can detect memory-management problems. It checks all read/write operations on memory and intercept the calls to malloc/new/free/delete.

- Compile a program to allow the compiler to collect the debugging information

  ```
  $gcc -g<option_number> -o progx progx.c
  ```

- The <option_number> denotes amount of debugging information collected (1 = min, 2 = default, 3 = max).
- Running the Memcheck tool of Valgrind and create a log file

  ```
  $valgrind --tool=memcheck --leak-check=full
  --log-file=progx_memcheck.log ./progx
  ```

- As Memcheck tool is the default in Valgrind and mentioning the log file is optional, alternatively

  ```
  $valgrind --leak-check=yes ./progx
  ```

Memcheck – An example

1. `$valgrind --tool=memcheck --leak-check=full --log-file=progx_memcheck.log ./progx`
2. `$vi progx_memcheck.log`

```
...
==9202== HEAP SUMMARY:
==9202== in use at exit: 0 bytes in 0 blocks
==9202== total heap usage: 1 allocs, 1 frees, 1,024 bytes allocated
...
==9202== LEAK SUMMARY:
...
```

Memcheck – Understanding the error summary

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
int main(){
    char *source ="computing"; // Allocates 10 bytes
    char *copy =(char *)malloc(strlen(source));
    strcpy(copy, source); // source is larger than copy
    printf("%s", copy);
    free(copy);
    return 0;
}
```

1 $vi progx_memcheck.log

```
==4651== Invalid write of size 1
==4651== at 0x80486A4: main (progx.c:7)
...
```

Memcheck – Understanding the error summary

```
#include<stdio.h>
#include<stdlib.h>
int main(){
    int *p=(int *)malloc(10 * sizeof(int));
    p[10] =0; // Heap block overrun
    free(p);
    return 0;
}
```

1. $vi progx_memcheck.log

```
==4651== Invalid write of size 4
==4651== at 0x80486A4: main (progx.c:5)
...
```

Memcheck – Understanding the leak summary

- **definitely lost** – memory is lost for sure (should be fixed!!!)
- **indirectly lost** – memory for the root node of a tree is free but not the rest of it (should be fixed!!!)
- **possibly lost** – memory is actually lost due to reasons like function allocates a buffer and returns it but the caller never frees the memory or a lot of allocated memory is actually not in need (should be fixed!!!)
- **still reachable** – memory is probably still in use at the end of the program (not a serious issue!!!)

Memcheck – Understanding the leak summary

```
#include<stdio.h>
#include<stdlib.h>
int main(){
    int i, *a;
    for(i=0; i <10; i++)
        a =(int *)malloc(sizeof(int) * 100);
    free(a); // Freeing a only once is problematic
    return 0;
}
```

1. $vi progx_memcheck.log

```
==24810== LEAK SUMMARY:
==24810== definitely lost: 3,600 bytes in 9 blocks
==24810== indirectly lost: 0 bytes in 0 blocks
==24810== possibly lost: 0 bytes in 0 blocks
==24810== still reachable: 0 bytes in 0 blocks
```

Memcheck – Understanding the leak summary

```
#include<stdio.h>
#include<stdlib.h>
int main(){
    int i, *a;
    for(i=0; i <10; i++){
        a =(int *)malloc(sizeof(int) * 100);
        free(a); // Freeing a repeatedly works well
    }
    return 0;
}
```

1. $vi progx_memcheck.log

```
==24810== HEAP SUMMARY:
==24810== in use at exit: 0 bytes in 0 blocks
==24810== total heap usage: 10 allocs, 10 frees, 4,000 bytes allocated
==24810== All heap blocks were freed – no leaks are possible
```

Memcheck – Understanding the leak summary

```
#include<stdio.h>
#include<stdlib.h>
int main(){
    int i, *a[10];
    for(i=0; i <10; i++){
        a[i] =(int *)malloc(sizeof(int) * 100);
        free(a[i]); // Freeing a[i] each time works well
    }
    return 0;
}
```

1. $vi progx_memcheck.log

```
==24810== HEAP SUMMARY:
==24810== in use at exit: 0 bytes in 0 blocks
==24810== total heap usage: 10 allocs, 10 frees, 4,000 bytes allocated
==24810== All heap blocks were freed – no leaks are possible
```

Memcheck – Understanding the leak summary

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
int main() {
    char *source ="computing";
    char *copy =strdup(source);
    copy ++, abort(), copy --;
    free(copy);
    return 0;
}
```

1 $vi progx_memcheck.log

```
==24810== LEAK SUMMARY:
==24810== definitely lost:  0 bytes in 0 blocks
==24810== indirectly lost:  0 bytes in 0 blocks
==24810== possibly lost:  10 bytes in 0 blocks
==24810== still reachable:  0 bytes in 0 blocks
```

Cachegrind

The *Cachegrind* is a cache profiler. It performs detailed simulation of the I1, D1 and L2 caches in your CPU and so can accurately pinpoint the sources of cache misses in your code. It identifies the number of cache misses, memory references and instructions executed for each line of source code, with per-function, per-module and whole-program summaries.

- Running the Cachegrind tool of Valgrind and create a log file

```
$valgrind --tool=cachegrind --log-file=
progx_cachegrind.log ./progx
```

- Open the log file

```
$vi progx_cachegrind.log
```

Cachegrind – An example

1. `$valgrind --tool=cachegrind --log-file= progx_cachegrind.log ./progx`
2. `$vi progx_cachegrind.log`

...
–9411– warning: L3 cache found, using its data for the LL simulation.
==9411== I refs: 264,449
==9411== I1 misses: 981
==9411== LLi misses: 967
==9411== I1 miss rate: 0.37%
==9411== LLi miss rate: 0.37%
...

Callgrind

The *Callgrind* is an extension to Cachegrind. It provides all the information that Cachegrind does, plus extra information about callgraphs.

- Running the Callgrind tool of Valgrind and create a log file

```
$valgrind --tool=callgrind --inclusive=yes
--tree=both --log-file=progx_callgrind.log ./progx
```

- Open the log file

```
$vi progx_callgrind.log
```

Massif

The *Massif* is a heap profiler. It performs detailed heap profiling by taking regular snapshots of a program's heap. It produces a graph showing heap usage over time, including information about which parts of the program are responsible for the most memory allocations.

- Running the Massif tool of Valgrind and create a log file

```
$valgrind --tool=massif --log-file=
progx_massif.log ./progx
```

- Open the log file

```
$vi progx_massif.log
```

Helgrind

The *Helgrind* is a thread debugger which finds data races in multithreaded programs. It looks for memory locations which are accessed by more than one (POSIX p-)thread, but for which no consistently used (pthread mutex) lock can be found. Such locations are indicative of missing synchronisation between threads, and could cause hard-to-find timing-dependent problems.

- Running the Helgrind tool of Valgrind and create a log file

```
$valgrind --tool=helgrind --log-file=
progx_helgrind.log ./progx
```

- Open the log file

```
$vi progx_helgrind.log
```